

Software Design Description

Camfranglais

Public analysis. Immutable tests. Read-only reference resources.

DETERMINISTIC / PUBLIC ACCESS / EVIDENCE-LED

Source-grounded design of the final Camfranglais compiler workspace. The public application requires no account and retains completed analysis tests. Collection, saved grammar and existing recordings are protected from public edits. Retained operator/desktop code is distinguished from active public functionality. Software behavior and collection review flags do not certify human fieldwork.

AUTHOR GROUP

Name	Matricule
Kwete Ngouba Junior Rayan	ICTU20241377
Djemtchimo Noukui Bruno Jonatan	ICTU20241585
Amina Boubakary	ICTU20241870

Version 4.1 | 27 September 2026

Faculty of Information and Communication Technologies

Instructor: Engr. Tanwi Nkiamboh

Assignment due: 29 September 2026

Contents

1	Document Status and Design Scope	5
1.1	Baseline and audience	5
1.2	Three explicit application modes	5
1.3	Evidence and provenance boundary	5
2	Architecture and Decisions	6
2.1	System composition	6
2.2	Decision record	6
2.3	Implementation responsibility map	6
3	Public Request Boundary	7
3.1	Bootstrap, not login	7
3.2	Public route contracts	7
3.3	Forbidden, absent and throttled requests	8
4	Analysis Recording, Retry and Reporting	9
4.1	Successful append	9
4.2	Idempotency and stale grammar	10
4.3	Snapshot invariants	10
4.4	Analysis statistics	10
5	Complete Class Model and UML Conventions	10
5.1	Inventory and source verification	11
5.2	How to read the diagrams	11
5.3	Detailed model decomposition	11
6	Deterministic Compiler Design	12
6.1	Automata and their implementation boundary	12
6.2	Lexical pipeline	12
6.3	Grammar and predictive parsing	12
6.4	Supplied case interpretation	13
7	Storage, Ownership and Recovery	13
7.1	Persistent structures, not invented objects	13
7.2	Request-local consistency	14
7.3	Private maintenance and compatibility	14

8 Frontend and Recorded Media	15
8.1 Navigation and data ownership	15
8.2 Existing recorded voice only	15
9 Deployment, Trust and Operational Limits	16
9.1 Target host topology	16
9.2 Production settings and readiness	16
9.3 Failure and resource behavior	16
10 Verification and Traceability	17
10.1 Source and documentation checks	17
10.2 Dated application validation and known limitation	17
10.3 Design-to-contract index	17
A Full-size UML and Automata Atlas	18

List of Figures

1	Public application components and retained boundaries	19
2	Target deployment and protected persistent storage	20
3	Public use cases and private operational responsibilities	21
4	Overall relationships: public storage and immutable test results	22
5	Public-access state, read-only policy and ownership contracts	23
6	Shared store hierarchy and request-local storage protocols	24
7	Immutable snapshot models and response enrichment	25
8	Lexical statistics and UNKNOWN-only vocabulary approval	26
9	Grammar transformation and predictive-parse snapshot models	27
10	Saved-test summaries, aggregates and independent verdict totals	28
11	Public analyzer inputs and retained Collection mutation inputs	29
12	Lexical preview, health and metadata response contracts	30
13	Separate Collection, Dictionary and synthetic example models	31
14	Existing reading lookup and retained recording input models	32
15	Actual compiler runtime classes and function dependencies	33
16	Server settings, web boundary and shared throttling	34
17	The frontend runtime class, function components and hooks	35
18	Retained account classes: explicit legacy mode only	36
19	Retained history inputs and private backup contracts	37
20	Saved profile reuse and retained coursework/import classes	38
21	Retained local desktop collector and recorder classes	39
22	Public analysis, grammar guards and immutable test append	40
23	Read-only reference browsing and provenance-preserving handoff	41
24	Explicit private restore with authorization and a safety copy	42
25	Browser test-submission and uncertain-completion states	43
26	Retained desktop audio readiness and retry states	44
27	Public bootstrap without an account or login	45
28	Successful recording of a new complete public test	46
29	Idempotent retry and stale-grammar request handling	47
30	Deterministic grammar, lexical and predictive-parser calls	48
31	All-test aggregation with paginated summaries	49
32	Dictionary search and unchanged reference-text handoff	50
33	Existing recorded-reading lookup and guarded audio streaming	51
34	Forbidden public writes, hidden APIs and request limits	52
35	Configured private automatic-backup maintenance	53

36	Finite-state model equivalent to the token segmentation regex	54
37	Deterministic pushdown control for the saved LL(1) grammar	55

1 Document Status and Design Scope

1.1 Baseline and audience

This design describes source revision **a1b88aa**, reviewed for the final documentation on 27 September 2026. It is intended for implementers, reviewers and the server operator. Dated corpus, test and deployment observations are identified explicitly; neither a source revision nor a diagram proves that a new release has been installed on the public server.

The application is a deterministic lexical/CFG workbench, not an identity portal, chatbot, translation generator or generic dashboard. Its five navigation pages are **Franç Analyzer**, **Analysis**, **Collection**, **Dictionary** and **Synthetic examples**. The current UI's Analyze action records a complete public test. Reference browsing, text handoff and recording playback do not add Collection entries.

1.2 Three explicit application modes

Factory selection	Implemented composition
<code>require_auth=None</code>	Default public mode. Installs public-access policy, shared storage, read routes and analyzer-test append. Uses ServerSettings ; does not install authentication, workspace management, import, coursework or backup APIs.
<code>require_auth=True</code>	Explicit retained authenticated API. Installs legacy identity and maintenance routes and requires AuthSettings . Its workspace context is now shared; this is not a return to separate editable public projects.
<code>require_auth=False</code>	Unrestricted local maintenance compatibility. Not the public product. The factory rejects this choice in production. Unbound legacy dataset/profile storage remains relevant only to explicit local workflows.

The design assumes the normal `include_academic=True` factory setting. Disabling academic routes also disables the bootstrap's saved-test capability; the current five-page frontend deliberately requires that capability. Optional factory flags are not a promise that the same UI works in every maintenance configuration.

1.3 Evidence and provenance boundary

The observed Collection contains 13 entries: 12 supplied sentences and one word, with 20 retained revisions. The classified source has 878 supplied vocabulary rows; merging distinct legacy senses produces 938 Dictionary entries. There are 26 clearly labelled synthetic examples. These are distinct populations, not 938 collected statements or 26 additional field observations.

The original 28 saved tests were preserved. A 26 September local observation found 29 after an additional public test had been recorded. Read-only public browser captures on 27 September show 34 saved tests and 13 Collection entries. Counts grow with use; none of these dated totals is a permanent application constant. Tests may include repeated text and rejected results and are separate from Collection and the report's fixed twelve-case evaluation.

Human collection location, date, consent and transcription authenticity have not been confirmed. The historical `compiler/parser/yaounde.py` filename identifies an implemented case-set gram-

mar, not independently verified fieldwork in that location. Original statements, including unresolved forms, must not be silently corrected to improve acceptance statistics.

2 Architecture and Decisions

2.1 System composition

Figure 1 separates the browser, FastAPI boundary, deterministic compiler functions and protected persistence. FastAPI serves the built React application and API on the same origin. The production host uses a reverse proxy; Vite is a development tool, not the production web server.

Most behavior is implemented as Python module functions and React function components/hooks. The class model therefore documents actual data models, storage/policy objects, compiler helper classes and retained desktop classes. It does not turn functions into invented controllers, services or repositories.

2.2 Decision record

1. **Public access without a fabricated identity.** An opaque browser cookie and CSRF token protect submissions, but neither is an account. The stable internal public actor UUID supports saved-test idempotency. No fake user row, account counter or registration step is introduced.
2. **One shared dataset, narrower public writes.** `PublicWorkspaceStore` extends `SharedWorkspaceStore`, which extends `WorkspaceStore`. The public subclass permits only a new analyzer test. It rejects `Collection/grammar/recording` mutations, existing-test writes and imports. Hiding a button is not the enforcement mechanism.
3. **Deterministic and inspectable results.** `Regex/lexicon` rules, explicit CFG transformations and a bounded predictive parser produce token tables, sets, conflicts and traces. No provider decides acceptance, generates a voice or writes coursework.
4. **Snapshot evidence rather than mutable saved results.** Each recorded test stores its exact text, grammar source, lexer/parser results and grammar analysis. Reopening a test does not rerun it against today's `Collection` or `grammar`. Public append does not imply public update or deletion.
5. **Two meanings of acceptance remain separate.** Vocabulary approval rejects exactly inputs with `UNKNOWN` tokens. CFG acceptance separately reports whether the token-category stream can be parsed by the submitted saved grammar. Neither is a judgement about a speaker.
6. **Compatibility is preserved, not advertised as active UI.** Private accounts, old workspaces, saved histories, coursework archives and desktop recording code remain. Their classes are covered, but their private routes and legacy workflows are not presented as public features.

2.3 Implementation responsibility map

Source area	Responsibility
backend/main.py, config.py, web.py	Mode composition, server settings, static assets, health, host/TLS policy, request-size limits and response headers.

Source area	Responsibility
backend/public_access.py, rate_limits.py	Public allowlist, browser request protection, shared context, rate limits, read-only policy and saved-grammar guard.
backend/analyzer*.py, token_statistics.py	Analyzer state, deterministic runs, strict snapshot models, idempotent recording, saved-test inspection and aggregate token statistics.
backend/workspaces.py, shared_workspace.py, ownership.py	SQLite persistence, request-local binding, ownership contracts, shared migration and retained recovery behavior.
backend/collection.py, dictionary.py, examples.py	Collection reads and separate source-labelled Dictionary/synthetic data. Collection mutations are retained but not mounted publicly.
backend/readings_api.py, readings_models.py, audio_api.py	Existing recorded-reading lookup, typed recording records and guarded media responses; upload/change/remove routes are private compatibility.
compiler/lexer, compiler/parser	Segmentation, classification, vocabulary/reference rules, phrase/language annotations, CFG notation/transforms, sets, table and parser.
backend/coursework_store.py, coursework_models.py	Saved grammar/profile model and storage protocol reused by the analyzer; profile/screenshot/report-generation routes remain outside public scope.
backend/auth.py, workspace_backups.py, import*.py	Retained identity/local extraction and private operator backup/recovery. Background backup maintenance may run without exposing backup APIs.
frontend/src, data_collector	Current public React UI and optional retained local desktop collector; these are different execution paths, not two names for public recording.

3 Public Request Boundary

3.1 Bootstrap, not login

On startup, `PublicSession` calls `GET/api/public/session`. The successful default response contains `access_mode=public_read_only`, a `csrf_token`, and capabilities `analyze=true`, `save_tests=true`, `edit_collection=false`, `edit_grammar=false` and `edit_recordings=false`. The frontend validates this contract before rendering `App` and stores the CSRF token only in its API module state. Figure 27 shows this interaction.

The `camfranglais_public` cookie is an opaque random value, not an identity. It has `HttpOnly`, `SameSite=Lax`, `Path=/` and, in production, `Secure`. The server reuses a valid cookie and replaces an invalid or missing value. It derives a CSRF token and compares tokens in constant time. No email address, display name or signed-in identity is created.

3.2 Public route contracts

Route/method	Contract and effect
GET/api/health	Returns compiler health. Does not initialize or consume a shared request-limit window. Health alone does not prove successful public bootstrap.
GET/api/public/session	Prepares the public cookie/token/capabilities; no account creation or login.
GET/api/metadata	Collection labels, entry types, lexical categories and dataset languages.
GET/api/dataset	Searches Collection without modifying it. Counts cover the whole Collection, not only matches. Public entry ownership is redacted.
GET/api/dictionary, GET/api/examples	Source-labelled reference/synthetic results with query, offset and limit. Each response page is bounded to at most 100 entries.
GET/api/analyzer	Saved grammar, lexical specification and Collection counts; no profile or grammar editing is exposed.
POST/api/analyze	Non-persisting lexical result. AnalyzeRequest requires nonblank text of 1–4000 characters and preserves its original whitespace.
POST/api/analyzer/analyze	Non-persisting full analysis against the saved grammar, with corpus analysis. This API is not the UI's saved-test submission action.
POST/api/analyzer/tests	Records an immutable test, or returns an identical earlier request result. Accepts canonical request UUID, saved grammar and 0–4000 characters of exact test input. An empty test is distinct from a valid Collection entry.
GET/api/analyzer/tests	All-test aggregates plus a bounded page of saved summaries. Vocabulary and grammar acceptance totals are separate.
GET/api/analyzer/tests/ /id	Returns one stored snapshot with derived approval and redacted ownership; does not recompute it.
POST/api/readings/lookup	Looks up an existing recording by normalized text and language. Returns reading=null when absent.
GET/api/readings/id/audio, GET/api/dataset/id/audio	Streams a checked existing audio file for playback/download; does not synthesize, capture or upload sound.

All four allowed POST paths require an allowed **Origin**, the public cookie and **X-CSRF-Token**. In production the allowed origin is the configured public HTTPS origin. Development adds explicitly configured local origins. Route payload validation and domain bounds are additional checks, not replacements for Origin/CSRF validation.

3.3 Forbidden, absent and throttled requests

Figure 34 distinguishes the outcomes:

- Public writes targeting Collection, grammar or recordings return **403**. The store also rejects forbidden writes and all imports.
- Authentication, workspace management, coursework, imports, backups, legacy history and restore endpoints are unavailable publicly: **404**, not an account challenge or an HTML SPA success response.
- A wrong Origin or invalid/missing CSRF on an allowed POST returns **403** before domain work.
- The shared limiter permits 120 requests per 60-second IP window. Exhaustion returns **429** and **Retry-After**. The counter is in `public-access.sqlite3`, separately from `auth.sqlite3`.

The public limit covers permitted API requests other than health; static requests and preflight handling are not counted by this path. Shared-storage lock timeouts return a busy response rather than silently bypassing the lock. Unexpected storage failures are reported without returning private paths or account records.

The historical `X-Mboa-Project` header and `project` query parameter are validated if supplied, but they do not partition the public workspace or select another account. The current frontend has no project selector.

4 Analysis Recording, Retry and Reporting

4.1 Successful append

Figures 22 and 28 specify the normal path. The route uses `course_operation`, which holds the context-bound dataset lock around the saved-grammar check and recording work. This prevents a concurrent cooperating writer from changing the referenced revision halfway through the operation.

1. Compare the submitted canonical grammar with the saved profile's grammar. The request-local public flag activates this restriction.
2. Look for an earlier request mapping for the stable internal public actor. If one exists, compare exact input text and stored grammar source before reuse.
3. Analyze the grammar, read Collection and build the reviewed-word lexicon. Run deterministic lexical analysis and predictive parsing.
4. Construct and validate a `RecordedTest`. Matching-entry metadata is derived from exact text equality, not a fuzzy claim that the test was collected.
5. Generate the actor-scoped UUID5 identifier. The shared store authorizes a new analyzer-test write, inserts the snapshot and request mapping in one transaction, and increments the shared version.
6. Add response-only vocabulary approval and redacted ownership. The original persisted `RecordedTest` remains unchanged.

A valid analysis can retain a rejected parse. An invalid grammar or exceeded grammar-analysis work bound raises an explicit validation error before snapshot insertion. Public analysis never adds a Collection entry or modifies the saved grammar. The non-persisting analysis endpoints remain distinct API contracts.

4.2 Idempotency and stale grammar

Figure 29 documents an important ordering detail. `require_shared_grammar` normally rejects a changed submitted grammar with 409 and reload guidance. When that request ID already exists, it allows the recording function to perform its stricter exact-payload comparison. Consequently:

- A new request with a stale grammar is rejected before analysis.
- An existing request with identical text and grammar source returns the original snapshot, even if a curator subsequently changed the saved grammar.
- Reusing an ID with different text or grammar source returns 409 and never overwrites the earlier result.
- An ambiguous request mapping inherited from multiple former projects is reported as a conflict rather than selecting an arbitrary old test.

In the browser, `pendingTest` keeps the UUID, input and grammar across an uncertain retry. Success clears it. A later intentional Analyze click therefore creates a new test even for repeated text. Editing the draft discards the old pending retry key. Reloading the tab loses its unsent local draft/key but does not erase a test already committed on the server.

Stopping the UI request, navigating away or reaching a timeout does not guarantee server cancellation or rollback. The user is told to refresh Analysis and inspect saved tests before assuming nothing was recorded. Figure 25 models these browser states without inventing a runtime state-enum class.

4.3 Snapshot invariants

`SnapshotModel` uses strict validation and forbids extra fields. `LexicalSnapshot.consistent_statistics` checks statistics against its stored token sequence. `GrammarAnalysis.consistent_symbols` verifies start/nonterminal/set consistency and refuses an LL(1) claim with conflicts. `ParseResult.consistent_outcome` relates acceptance to the absence of an error. `RecordedTest.consistent_consumption` checks consumed length and requires an LL(1) grammar for an accepted parse.

These are real validation methods, not evidence that the DTOs are declared frozen. Public immutability is enforced by route composition and store policy. Figures 7, 8 and 9 expose the actual nested contracts.

4.4 Analysis statistics

The report traverses every retained test, holding only one full snapshot at a time. It computes token/category counts, raw and normalized frequencies, spelling variations, unknown-token review and recorded topic/language counts. Only summaries are paginated; changing the page does not change the aggregate population. An empty report has a null acceptance rate.

`summary` counts vocabulary acceptance; `grammar_summary` counts CFG acceptance. Each summary carries both results. Repeated intentional tests count repeatedly; the system does not silently deduplicate an experiment. Figures 10 and 31 show this design. Collection entry/type/review counts are a different statistic.

5 Complete Class Model and UML Conventions

5.1 Inventory and source verification

The inventory `docs/diagrams/class-coverage.json` covers all **98 authored production classes** found by `tools/check_documentation.py`. It includes Python protocols, private classes, `ReferenceEntry`, and the lexer `Token` generated through `namedtuple`. The authored TypeScript runtime class is `frontend.src.api.ApiError`. Tests, maintenance tools, external library types and erased TypeScript interfaces are excluded from this production count.

Pylance type-hierarchy facts confirm `PublicWorkspaceStore` $\rightarrow$ `SharedWorkspaceStore` $\rightarrow$ `WorkspaceStore`; source declarations and the checked inventory ground the remaining class and method names. Function modules, React hooks/components, grammar dictionaries, table rows and parse-internal dictionaries are not promoted into fictional OO wrappers.

5.2 How to read the diagrams

- Figure 4 provides a readable orientation view. The detailed related sheets, not a cramped monolithic class diagram, carry the complete inventory and selected actual members.
- Hollow triangles point to actual base classes. Dotted realization arrows identify structural protocol conformance, not nominal Python inheritance.
- Multiplicities are attached to stored or nested references. Inheritance, realization and function/module dependencies have no decorative cardinalities. Repeated DTO references are not exclusive-lifecycle compositions.
- String lengths and numeric bounds are constraints in braces, not multiplicities. The collection-valued types and numbered associations describe how many related values or objects may be held.
- **reference** means a class detailed on another sheet; **external** means a library type. Warm-toned retained classes identify compatibility code, not public capability.
- Selected operations are source operations; inherited Pydantic validation and serialization are described as inherited. Fields and generated/property members are labelled rather than invented as business methods.
- Every sequence call has an active sender, an immediately activated receiver, and a matching reply/deactivation. Resource activations indicate an interaction interval, not CPU execution by a file, database or person.
- Activity/state diagrams model guarded behavior and real logical states; they do not assert that every state is represented by a declared enum.

5.3 Detailed model decomposition

Figures 5, 6 and 16 document public policy, request-local contracts, actual stored references, deployment settings and shared rate limiting. The three storage adapters are `DatasetStorage`, `CourseworkStorage` and `OwnershipStore`.

Figures 11 and 12 distinguish public request/response contracts from retained Collection mutations. Figure 13 preserves the separation among Collection, Dictionary and synthetic practice. Figure 14 includes both active lookup/playback DTOs and the real but unmounted upload/consent models.

Figure 15 covers the four compiler runtime classes and their actual function dependencies. Figure 17 documents `ApiError` without pretending that React components are classes. Figures 18, 19, 20 and 21 preserve complete coverage of identity, old history/recovery, coursework/import and desktop code. `ProjectProfile` still participates in public saved-grammar loading despite the absence of public profile-editing routes.

6 Deterministic Compiler Design

6.1 Automata and their implementation boundary

Figure 36 is an equivalent finite-state model of `TOKEN_SPLIT_RE`, not an invented lexer class. Double circles accept a token form: *W* is word, *I* integer, *F* fraction, *P* punctuation and *O* another character. Non-accepting states *J* and *R* require a valid continuation after a joiner or decimal separator respectively. Missing transitions enter an omitted dead state. The scanner retains the last accepted prefix and resumes at the unconsumed character; whitespace is skipped between matches.

The word class *C* is the Unicode regex class `[~W\d_]`, not only ASCII letters. In particular, U+02BC belongs to *C*, so it follows word edges rather than competing with the pending-joiner edge. This distinction preserves the actual regex behavior and the diagram’s determinism. Classification by lexicon and morphology remains a later operation.

Figure 37 shows finite control for the LL(1) pushdown recognizer. Its stack initially holds the end marker and start symbol. One table entry replaces a top nonterminal with a reversed right-hand side; a matching terminal is popped; an epsilon production consumes no category. Acceptance requires the bottom marker and end-of-input together, not an arbitrary empty stack or any recognized vocabulary.

The family $q[a]$ encodes a single lookahead in finitely many states. The diagram buffers a category on its input edge; the implementation instead reads `categories[position]` and advances its position only when matching a terminal. This is an equivalent recognition model, not a claim that the Python function declares those control states. Input validation, non-LL(1) rejection and resource/repetition guards additionally bound the actual implementation. The project has not implemented an LR automaton.

6.2 Lexical pipeline

The lexer first segments raw input using a compiled regular expression. Internal apostrophes and hyphens remain part of words; a final non-whitespace fallback preserves unsupported input for classification rather than dropping it. Token text retains its original spelling.

`classify_token` applies structural regex rules, validated reviewed-word overrides, explicit existing lexicons, the supplied classified reference, additional French forms and finally documented morphological guesses. Multiple supplied categories yield `AMBIGUOUS`. Unsupported tokens yield `UNKNOWN`. Case/apostrophe normalization is used for lookup, not as a replacement for the displayed input.

Code-mixing annotations track compatible language evidence rather than arbitrarily assigning one language to every ambiguous word. Sentence-ending punctuation resets the run; unknown words carry no language evidence. Verb/slang phrase matches retain raw, source-ordered spans and do not collapse the parser’s token sequence.

The learned map comes from approved lexical annotations, not from every saved test. Known slang and supplied vocabulary are not rejected merely for being nonstandard French or English. `VocabularyApproval.from_statistics` rejects exactly a positive `UNKNOWN` count; this also means that zero tokens alone do not imply a vocabulary rejection. CFG behavior is separate.

6.3 Grammar and predictive parsing

The grammar reader validates explicit category-level notation. Productions are lists of symbols; an empty right-hand side represents epsilon. The transformation pipeline performs bounded ordered substitution, direct left-recursion elimination and factoring with before/after steps. It then

computes FIRST, FOLLOW and the predictive table.

Conflicting table cells are omitted from the unambiguous table while all competing productions remain visible in **conflicts**. Remaining recursion or table conflicts prevent predictive acceptance. The parser never selects a conflicting production arbitrarily. Its trace records stack, remaining categories and action before each action; the rightmost stack symbol is the top. \$ denotes end of input, not an ordinary input word.

Figure 30 shows the real function calls. The parser is bounded to 256 tokens, 1000 characters per token, 2000 trace steps and 1024 stack symbols. It also detects repeated stack/input configurations. A bound violation produces a specific rejected parse, not an unbounded loop.

The generic teaching starter in `compiler/parser/service.py` has an immutable JSON analysis cache decoded for each request. This is not the current saved assignment grammar in `compiler/parser/yaounde.py`; custom/saved grammars are analyzed afresh. The public client may use only the saved shared grammar, subject to the existing-retry exception already described.

6.4 Supplied case interpretation

The exact twelve fixtures in `compiler/tests/yaounde_cases.py` yield ten accepted parses and two rejections at token 6 in the supplied evidence. The unresolved forms `n'ais` and `alli` are preserved, not corrected to force twelve successes. These results characterize this lexer/grammar/case set; they neither prove universal Camfranglais validity nor authenticate the statements' human collection provenance.

7 Storage, Ownership and Recovery

7.1 Persistent structures, not invented objects

Physical structure	Design role
<code>shared-workspace/works pace.sqlite3</code>	Shared logical database. Base tables include metadata, projects, entries, revisions, history, coursework, screenshots and analyzer tests.
<code>analyzer_tests, test_requests</code>	Validated snapshot JSON and actor/request-to-test mappings. Current public IDs are deterministic for the stable actor/request pair. The mapping table also preserves migrated legacy mappings.
<code>ownership</code>	Original record kind/ID, owner UUID and owner name. Public responses substitute a redacted Ownership DTO without erasing these rows.
<code>legacy_profiles, migration_sources</code>	Preserved original profile context and migration tracking; not public profile or account-discovery endpoints.
<code>readings and shared-workspace/audio</code>	Normalized reading identity/metadata and generated-name audio files. Recordings are existing resources, never synthesis instructions.
<code>public-access.sqlite3</code>	Independent IP-window limits. It does not become an authentication database.

Physical structure	Design role
<code>auth.sqlite3</code> , legacy workspaces/archives	Retained private identity and historical data. The public API does not expose account records or old history/profile/archive management.
<code>shared-workspace/backups</code>	Private verified archives and recovery staging/catalogue metadata; no public backup download or restore route is mounted.

The shared root is not selected from an arbitrary browser path. A store object holds paths and a file lock; it does not retain a collection of live `RecordedTest` or `ProjectProfile` objects. SQL rows are validated into values when read. Class associations therefore do not pretend that database rows are in-memory object compositions.

7.2 Request-local consistency

`use_workspace` binds the same shared store to dataset, coursework and ownership `ContextVars` and resets the bindings on exit. Concurrent requests do not reassign a process-global CSV/profile directory. A store's file lock serializes cooperating shared operations, while SQLite transactions protect logical row changes and version increments.

Public ownership always serializes as `owner_id=null`, `owner_name=""`, `can_edit=false`. This is not a claim that every field of every public record is anonymous: original statement text, contributor context and other Collection metadata can still be public. Users are explicitly warned not to submit personal or confidential content in publicly retained tests.

7.3 Private maintenance and compatibility

The default lifespan may start `maintain_backups`. Each cycle checks private schedule metadata; automatic backup is disabled by default and runs only when configured and due. The shared maintenance store uses `system=True` and a separate internal system actor. It is not the public store and not a public administrator account.

The backup builder collects logical data and safe audio, checks referenced media, constructs a checksum manifest, validates the resulting archive and publishes it atomically before registering success. Failures are logged and reported through private state; they are not silently called successful backups. Figure 35 shows the configured background path.

Shared exports use document format 5, with format-4 compatibility. Base private workspace export uses format 3; older private formats 1 and 2 normalize during validation. The enclosing ZIP manifest remains format 1. These are retained format contracts, not publicly downloadable reports.

Figure 24 covers the explicit private restore procedure: bounded archive validation, authorization, expiring preview, version check, verified safety snapshot, staged media and transactional logical replacement. A non-system shared restore cannot alter another creator's records, request history or original archived profiles. The public subclass rejects every import. A post-commit cleanup error does not prove that the logical transaction rolled back; recovery must inspect saved state rather than retry blindly.

Audio staging and SQLite are coordinated but not one filesystem-wide ACID transaction. A workspace ZIP does not recreate identities or protected server configuration; operator disaster recovery must preserve those separately. The optional desktop collector and retryable recorder remain documented in Figures 21 and 26, without being confused with public browser functionality.

8 Frontend and Recorded Media

8.1 Navigation and data ownership

Public page	Behavior
Franc Analyzer	Exact typed or handed-off input, saved grammar status, explicit Analyze action, pending request identity and visible lexical/vocabulary/CFG results.
Analysis	All retained-test token statistics, independent vocabulary/grammar totals, paged summaries and full saved-snapshot inspection with transformations, FIRST/FOLLOW, LL(1) table/conflicts and parse traces.
Collection	Read-only supplied entries, metadata, review status, search and existing attachments. Copying an entry to analysis does not alter that entry.
Dictionary	Searchable classified vocabulary and distinct legacy senses with source document/line, gloss, origin and part-of-speech information.
Synthetic examples	Clearly constructed practice data, never presented as collected fieldwork; unchanged text may be handed to Franc Analyzer for an explicit test.

`PublicSession` replaces the earlier account gate. Account controls, registration counters and project selectors are absent, not merely styled out of sight. Report/presentation generation, import management, profile editing, screenshots and private history/recovery are not current public pages or APIs.

The API helper uses same-origin credentials, no-store requests, CSRF headers for non-GET submissions, bounded waiting and session-version guards against stale responses. Hooks separately manage pending/error state and cancellation. The saved-test call allows a longer 60-second wait; a timeout still cannot undo a server-side commit. Unsubmitted text stays in the current tab's React state, not a promised durable private draft.

Figure 23 and the Dictionary interaction in Figure 32 make the provenance-preserving handoff explicit. A synthetic/reference/Collection label follows the unchanged text; there is no hidden Collection insertion during browsing.

8.2 Existing recorded voice only

`useReadAloud` retains historical names such as `speak`, but now selects text/language for a stored-reading lookup. `ReadingLookup` normalizes case, apostrophe variants and whitespace into a language-qualified SHA-256 key. Punctuation and language remain significant.

A matching `ReadingView` leads to native audio controls and a download link. Audio access checks safe names, non-symlink files and containment in the protected audio directory. The response streams existing bytes. A missing reading yields `reading=null`; missing or unsupported audio produces an honest error. Browser autoplay failure falls back to manual play controls, not synthetic speech.

There is no public browser TTS/synthesis, microphone capture, transcription, voice cloning, upload, replacement or removal. The retained `AudioRecorder` function and `useAudioRecorder` hook

are not rendered in this workflow. Likewise, the retained `ReadingConsent` input is not retrospective proof of consent for every existing file. Figure 33 shows the current interaction.

9 Deployment, Trust and Operational Limits

9.1 Target host topology

Figure 2 specifies the intended Ubuntu 24.04 VPS at 109.199.120.38. Source and built assets live at `/var/www/francanglaiscompiler`; persistent data at `/var/lib/camfranglais`; restricted environment configuration at `/etc/camfranglais.env`. The `camfranglais.service` systemd service runs as the `camfranglais` user.

The application listens on private loopback 127.0.0.1:2021. Nginx serves ports 80/443 with TLS at <https://camfranglais.duckdns.org>. Port 2021 is not a public application URL. Proxy trust must be limited to the controlled reverse proxy so scheme, client address, secure-cookie policy and rate-limit keys remain meaningful. Private data/configuration must not be placed under frontend static assets.

9.2 Production settings and readiness

`ServerSettings` validates environment, absolute data directory and public origin. Public production requires a non-loopback HTTPS origin and a built frontend. SMTP and Google configuration are unnecessary for this default mode. `AuthSettings` inherits these settings and adds retained identity requirements only for the explicitly selected authenticated mode.

The web boundary applies API no-store behavior, content/type/frame/referrer protections, a same-origin CSP, URL/body bounds and production HSTS/host/TLS handling. A broad legacy microphone permissions-policy allowance is not proof that a public microphone control exists. Public mutation denial must be tested at the server, not inferred solely from UI screenshots.

Read-only checks on 27 September confirmed HTTPS bootstrap 200 with `access_mode=public_read_only` and the expected capabilities. The deployed interface opened directly into the five public pages without login; the genuine report screenshots retain that observation. This supersedes the older 26 September bootstrap-401 observation.

The documentation session did not perform a remote update or verify the operator's exact deployment steps. The diagram's internal host configuration, backup recovery and long-term availability are not certified by a browser capture. Full deployment acceptance must still exercise the expected allowed and denied routes and operational recovery, not health alone.

9.3 Failure and resource behavior

General API JSON bodies are bounded to 1 MiB in the web boundary; analyzer grammar/input contracts and parser work bounds impose tighter semantic limits. Pydantic rejects extra or inconsistent snapshot fields. Lock contention, missing media, malformed grammars, stale revisions, expired previews and rate-limit windows have explicit failure responses or private operational errors, rather than fabricated successful outcomes.

The application is a durable single-host design, not a horizontally distributed database service or end-to-end encrypted corpus. Server operator access and off-host recovery remain trust concerns. CSRF tokens protect browser submissions against cross-site requests; they do not turn public tests into confidential data.

10 Verification and Traceability

10.1 Source and documentation checks

The strict documentation checker derives the production class inventory from Python ASTs and TypeScript source declarations, checks every inventory-to-class mapping, balances sequence activations/calls/replies, and verifies source, PNG, atlas-page and SDD-page agreement. The design contains **37 diagram sheets**: 35 UML views (18 class, nine sequence, three activity, two state, and one each of component, deployment and use-case) plus two formal automata. The latter use conventional automata notation, including double-circle acceptance, and are rendered through PlantUML's local Graphviz support.

Diagrams are rendered locally with the repository's PlantUML JAR and Java, using an 8192-pixel canvas cap. No code, corpus or diagram source is sent to an external rendering service. The atlas gives each complete image its own proportionally scaled full-size sheet, without forcing complex models into A4 thumbnails. The A4 narrative and overview sheets provide the reading guide.

The common document build subsequently compiles the atlas and SDD and checks embedded-image fingerprints, page mappings, unresolved references and layout warnings. Source/activation checks alone are not a claim that an unbuilt PDF or the remote application is current.

10.2 Dated application validation and known limitation

The supplied 26 September evidence records 186 backend/launcher/archival tests, 148 frontend unit tests, 68 mocked browser tests, application/test TypeScript checks and a production frontend build passing. Fourteen distinct live desktop/mobile workflows were exercised successfully, but the full live run had two startup/teardown interruptions; the affected mobile checks later passed in a focused three-test rerun. This is not an uninterrupted 14/14 full-suite run.

The pre-existing `compiler.tests.test_parser.GrammarNotationTests.test_every_real_lexer_category_is_a_supported_terminal` still fails an old expected-category assertion. The documentation does not rewrite that expectation, hide the limitation or claim that all compiler tests or all CI are green. These application observations are separate from the diagram/class checks for this documentation revision.

10.3 Design-to-contract index

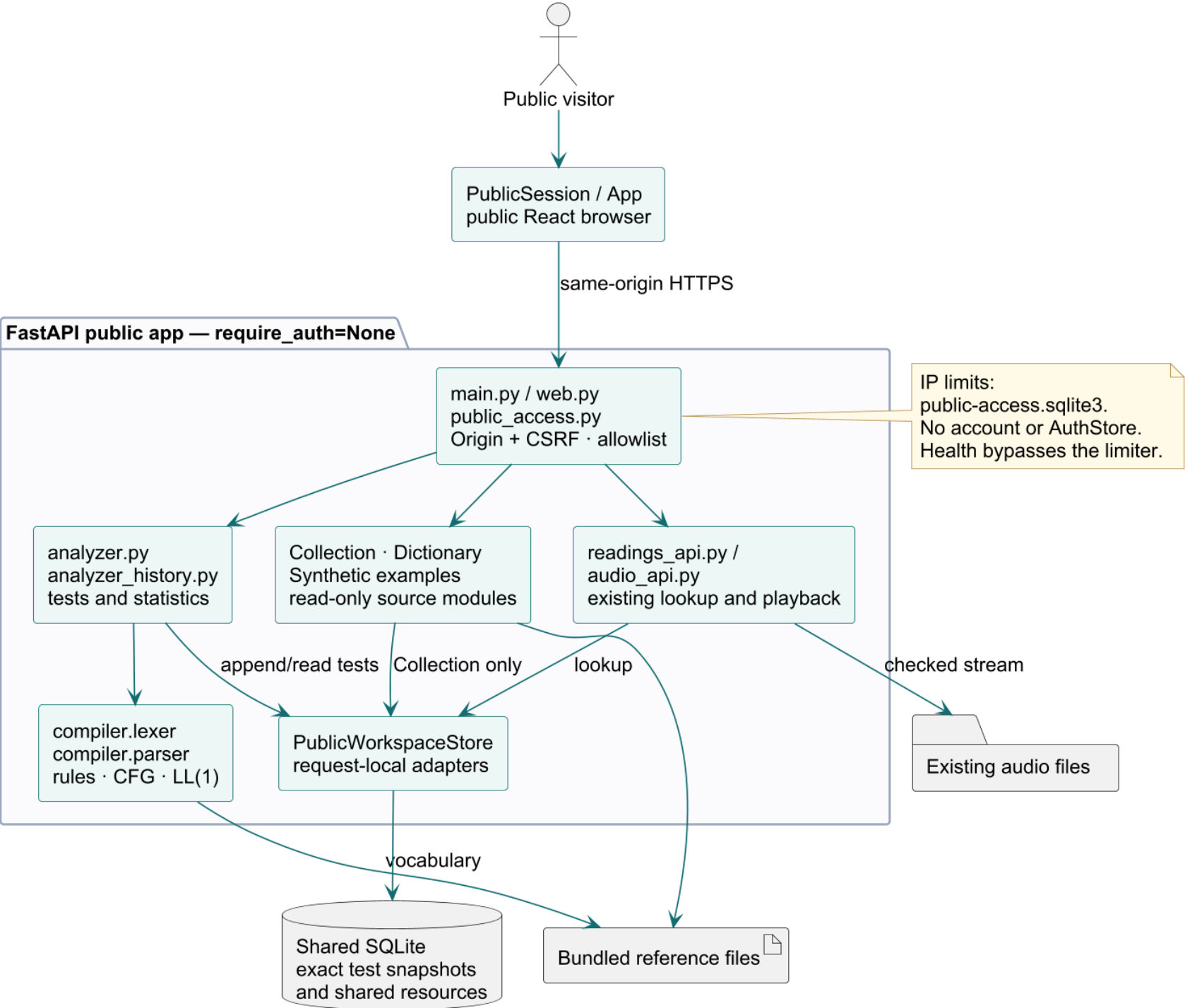
Contract	Primary implementation and diagrams
Public access, no fake account	<code>public_access.py</code> , <code>config.py</code> ; Figures 5, 27.
Read-only reference resources	Public allowlist and <code>authorize_write</code> ; Figures 3, 34.
Complete immutable test and retry	<code>analyzer_history.py</code> , <code>analyzer_models.py</code> ; Figures 28, 29.
Separate vocabulary/CFG verdicts	<code>VocabularyApproval</code> , parser analysis and <code>TestReport</code> ; Figures 8, 31.
Exact source and bounded compiler	<code>compiler/lexer</code> , <code>compiler/parser</code> ; Figures 15, 30, 36, 37.
Existing recording playback only	<code>readings_api.py</code> , <code>voice.ts</code> ; Figures 14, 33.

Contract	Primary implementation and diagrams
Preserved private compatibility	Retained account/workspace/import/desktop classes; Figures 18, 19, 20, 21.
Protected deployment/recovery	<code>web.py</code> , <code>workspace_backups.py</code> ; Figures 2, 35, 24.

A Full-size UML and Automata Atlas

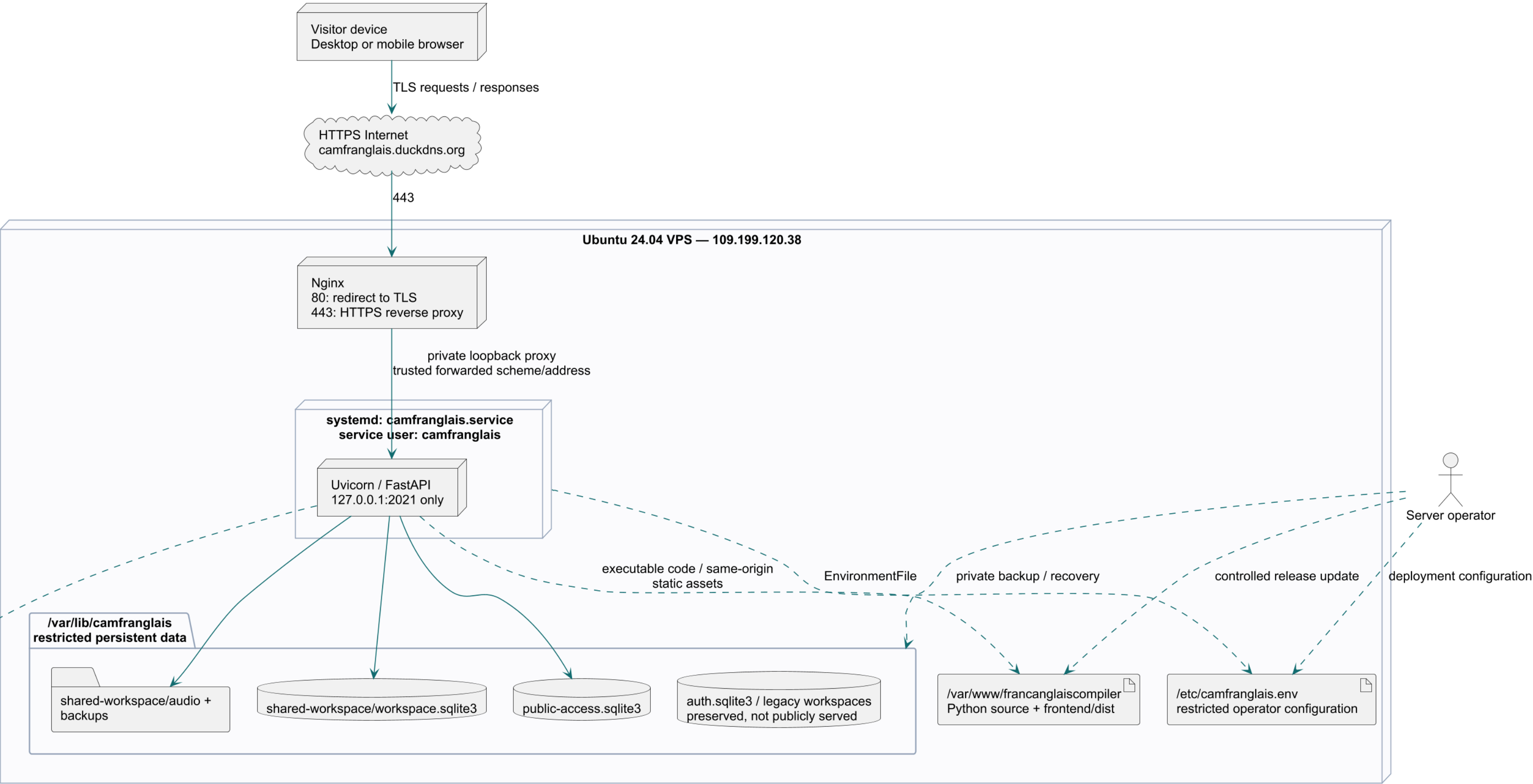
Each following sheet is the locally rendered image from the matching `docs/diagrams` PlantUML source. The ordering is identical to `docs/uml-atlas.tex`; captions and labels are part of this SDD. Retained-code sheets are intentionally included for complete production-class coverage, not as a list of public application features.

Camfranglais — public processing architecture

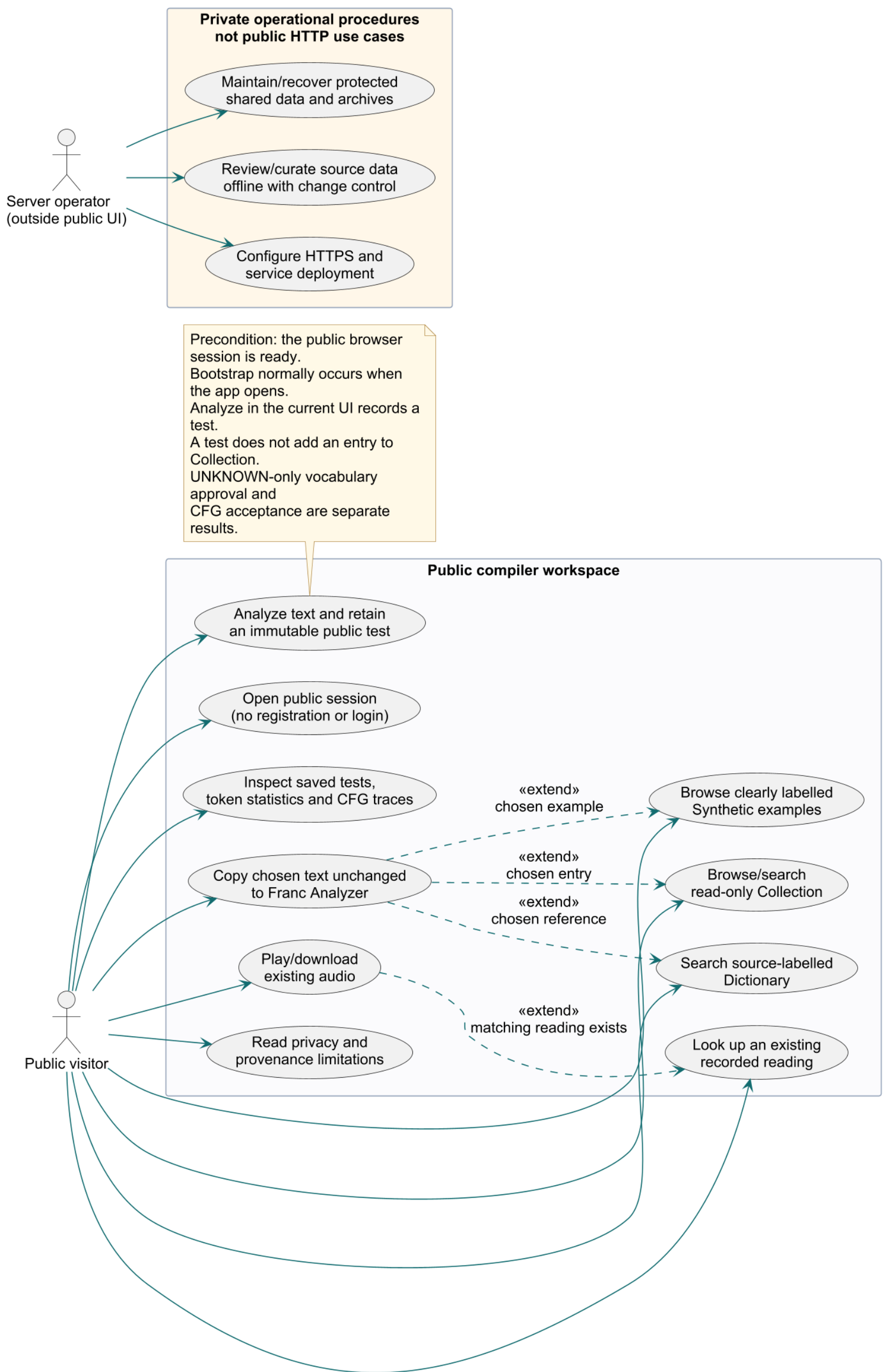


Components are modules, not invented runtime classes.
Only tests append; Collection, grammar and audio are read-only.
Operator backups/legacy APIs are outside this public UI.
No AI/voice generation, microphone or coursework export UI.

Camfranglais — target deployment and private persistence

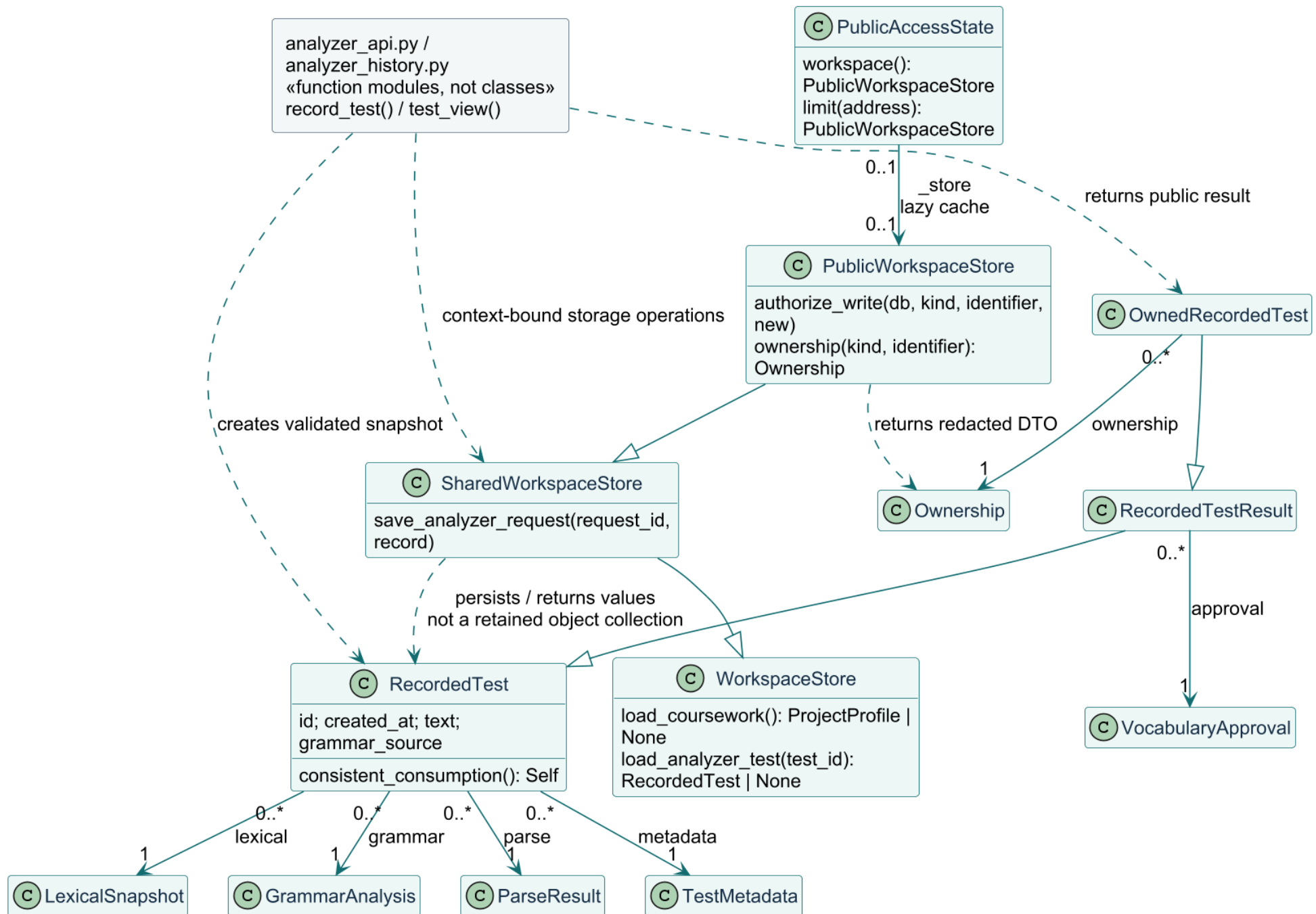


Camfranglais — public use cases, not account administration



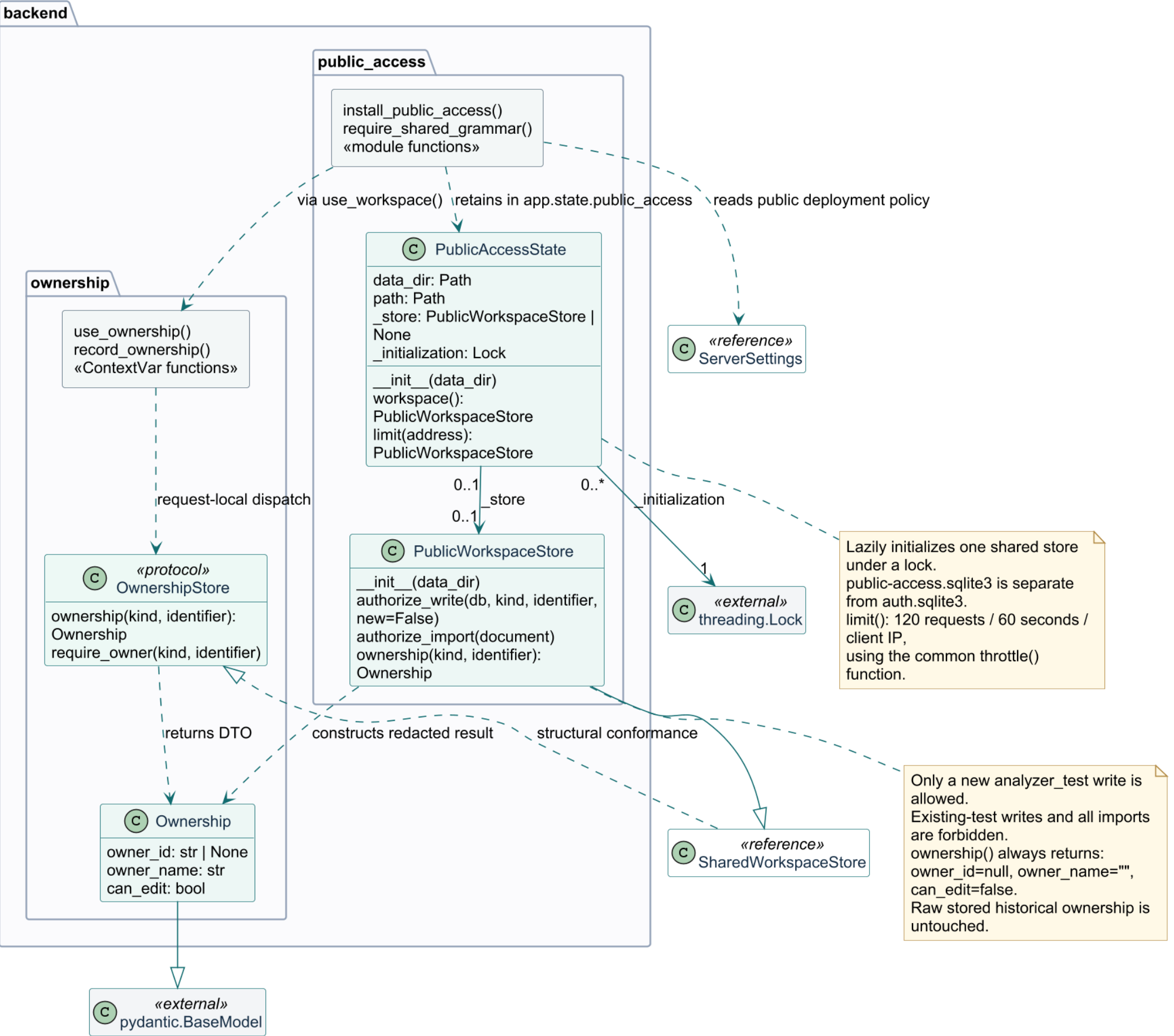
Read-only means protected reference resources; public test append is deliberately allowed.
No account controls, account counter, Collection/grammar/audio mutation or coursework export UI.
Recorded voice is playback, not browser synthesis, dictation or cloning.

Camfranglais — overall runtime relationships (orientation view)



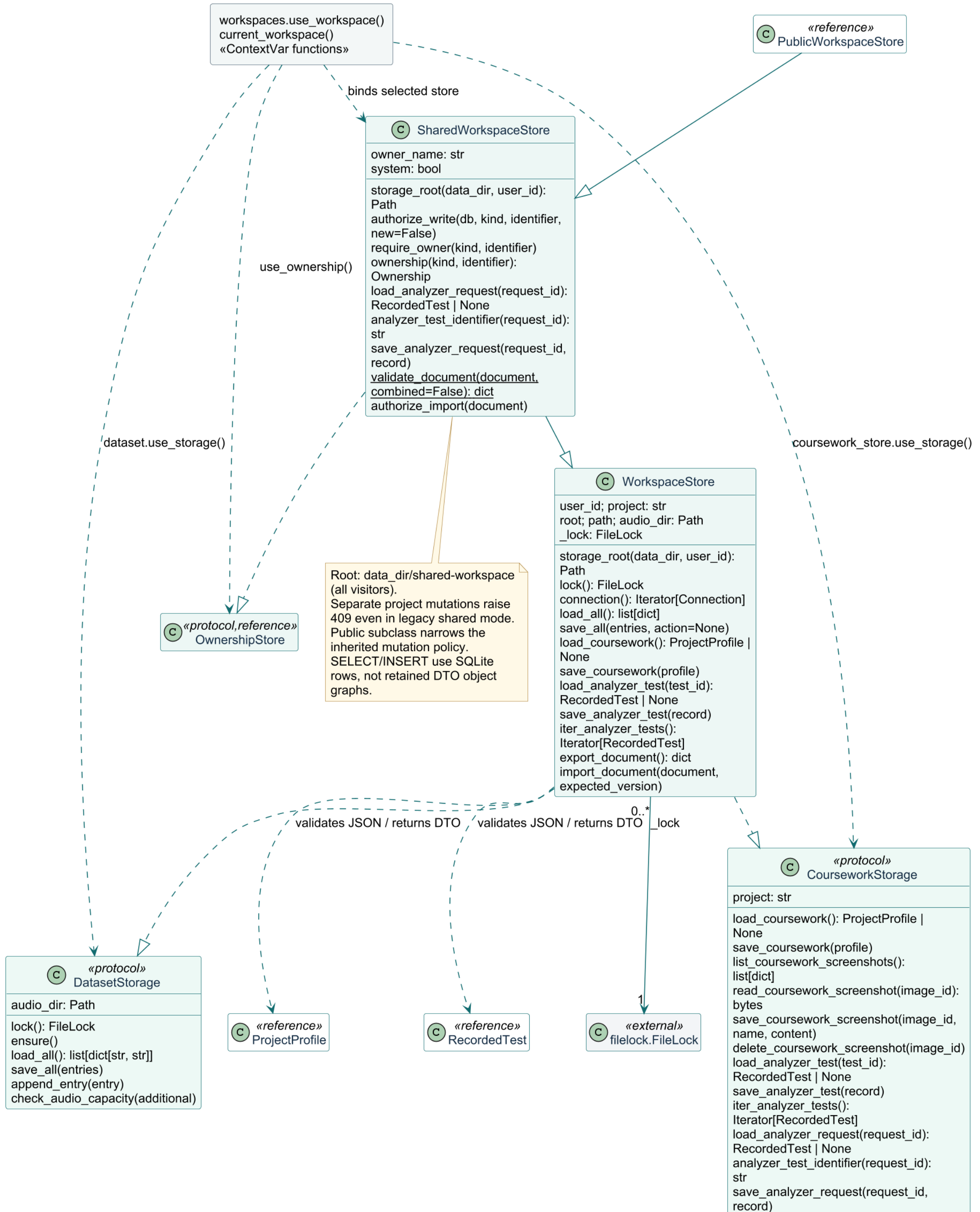
Orientation only: the detailed class sheets and class-coverage.json cover all 98 production classes. Hollow triangles: actual inheritance. Dashed arrows: usage dependencies, without multiplicities. Numbered arrows: actual stored/nested references; DTOs are not exclusive-lifecycle compositions. Public owner identity is redacted; the internal stable actor UUID is not a user account.

Public access — state, policy and ownership contracts



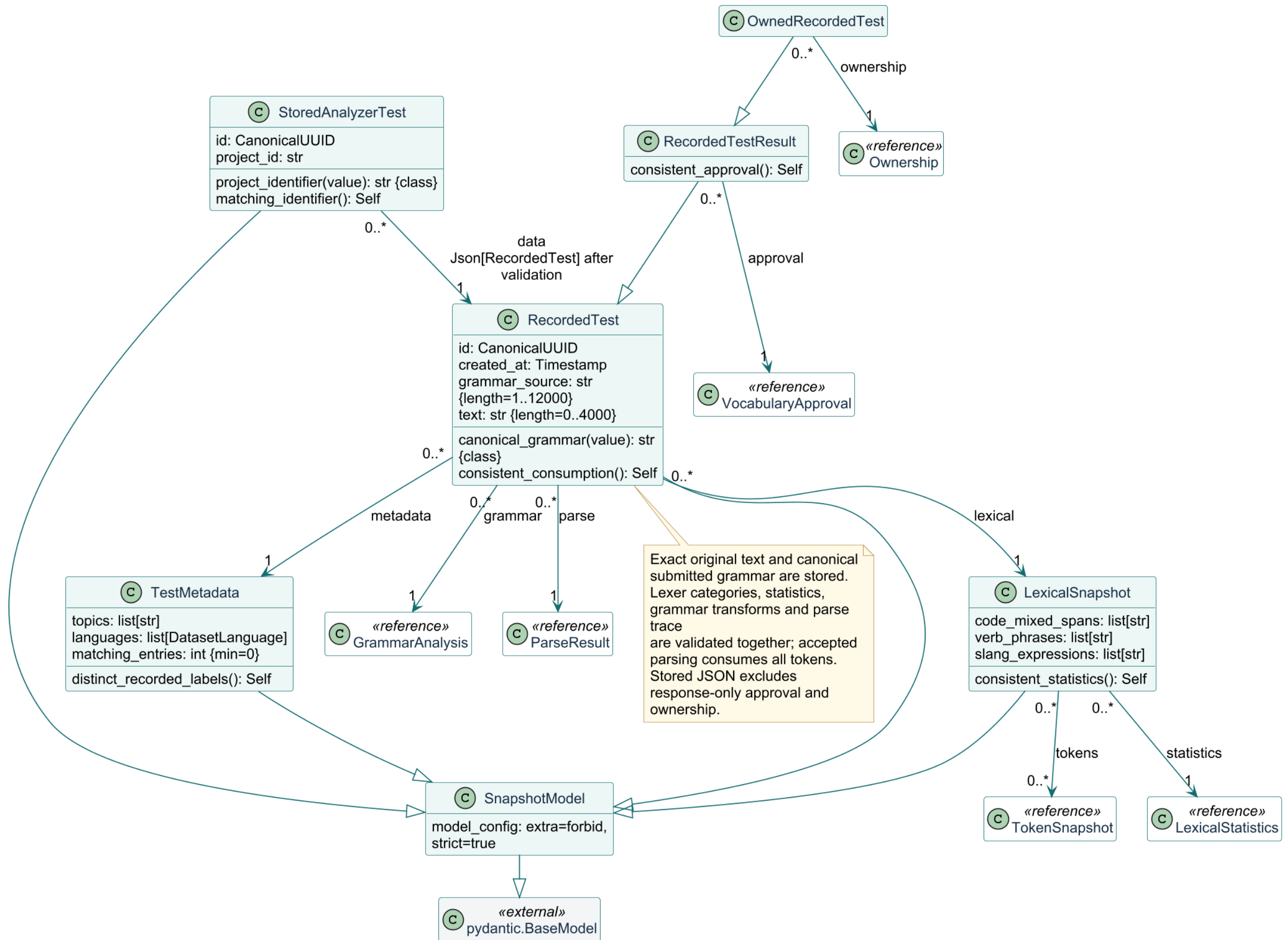
DatasetStorage and CourseworkStorage contracts are on class-workspaces.
Stable PUBLIC_OWNER_ID derives from UUID5; it supports request-id lookup, not authentication.
Opaque HttpOnly browser cookie + CSRF protect POST requests; no AuthStore or fake account is created.

Shared persistence — real store hierarchy and structural protocols



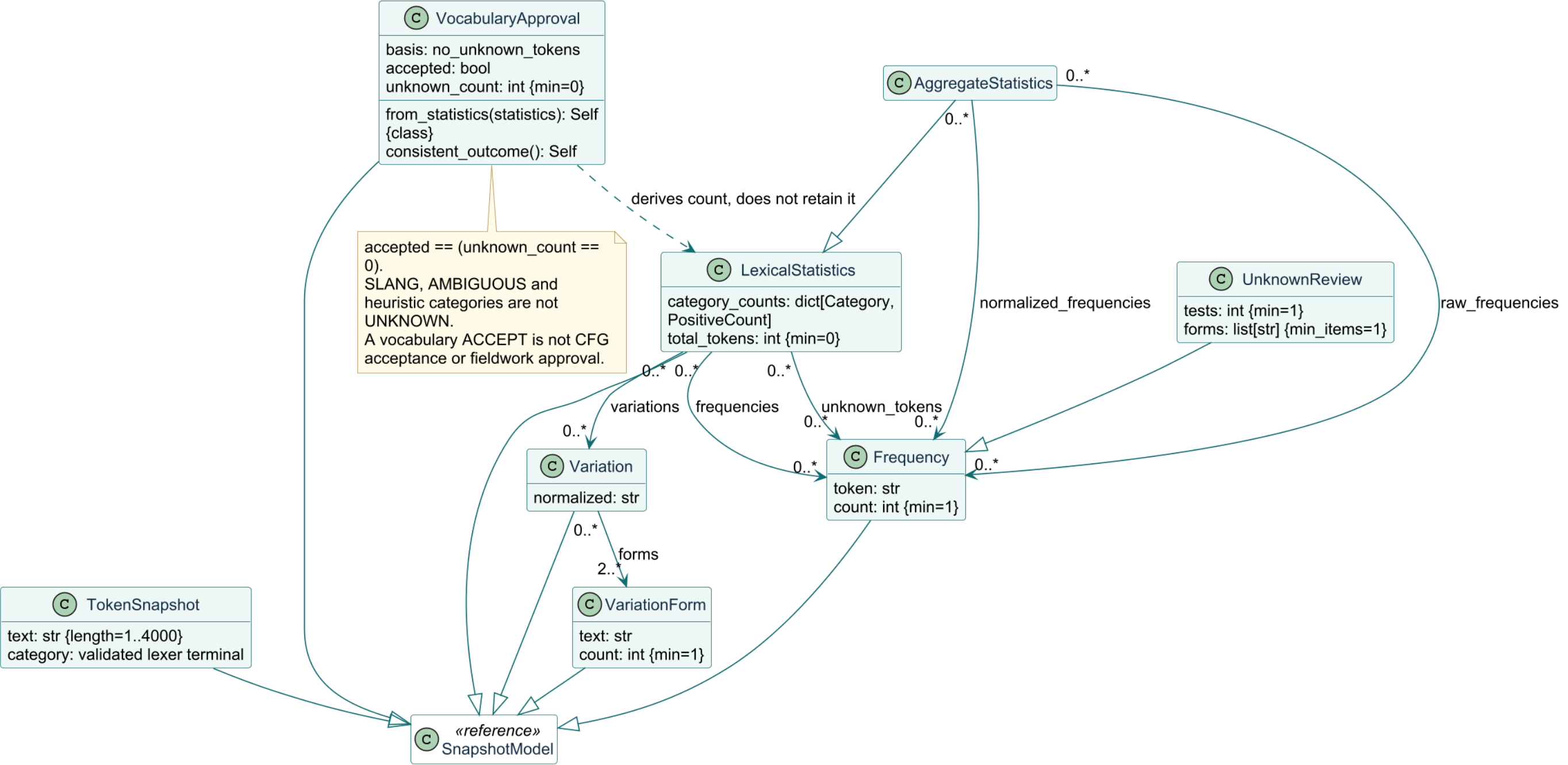
Hollow triangles toward bases; dotted triangles are structural Protocol conformance.
WorkspaceStore also retains revision/history/screenshot operations for private compatibility.
use_workspace() resets all request-local bindings on exit; no global file path is reassigned.
A per-store file lock serializes related shared reads/writes; SQLite transactions protect logical rows.

Immutable tests — persisted snapshots and public result enrichment



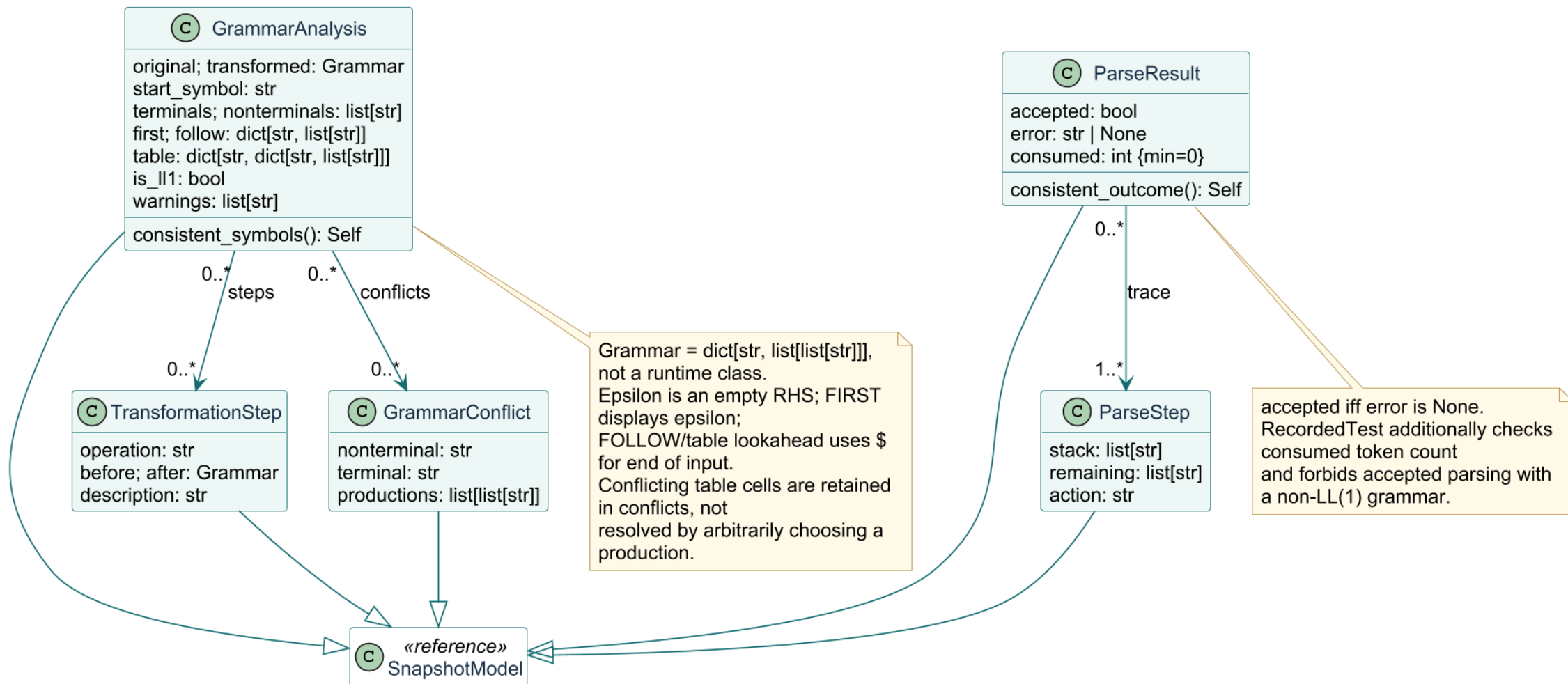
Source: backend/analyzer_models.py. DTO serialization/validation also uses inherited Pydantic methods. Public results add derived vocabulary approval and a redacted ownership DTO without rewriting snapshots. Immutability is a public persistence policy, not Python frozen=True on these Pydantic models.

Lexical snapshots, token counts and UNKNOWN-only approval



Source: backend/analyzer_models.py. All displayed generalizations are actual base classes. Token text retains spelling; casefolded and normalized frequency keys are separate projections. Counts/variati

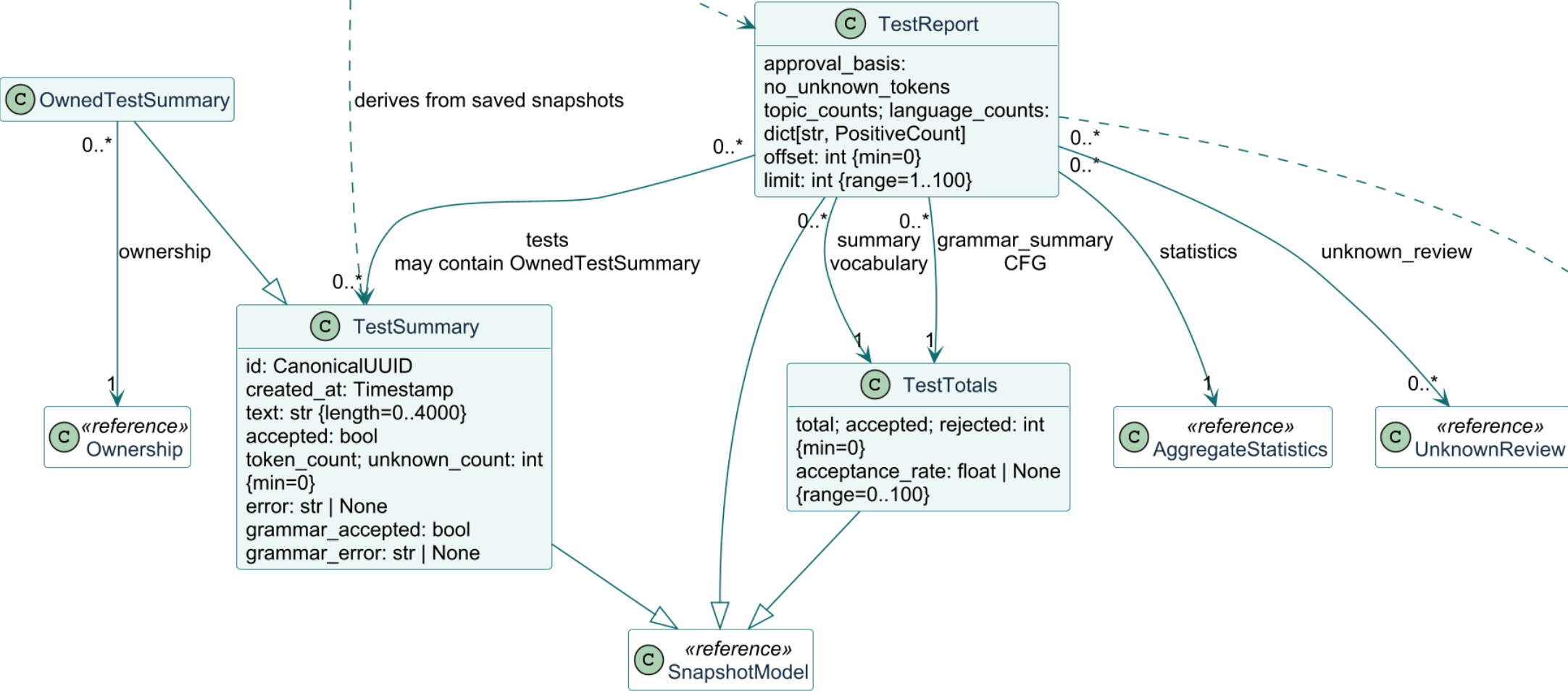
Grammar analysis and predictive parsing — snapshot DTOs



Source: backend/analyzer_models.py. These are real DTO classes, not the compiler's internal dictionaries. Compiler functions create dictionaries; snapshot validation constructs these typed nested values. GrammarAnalysis and ParseResult are separately referenced by RecordedTest, not associated with each other.

Saved-test reporting — two verdict totals and paged summaries

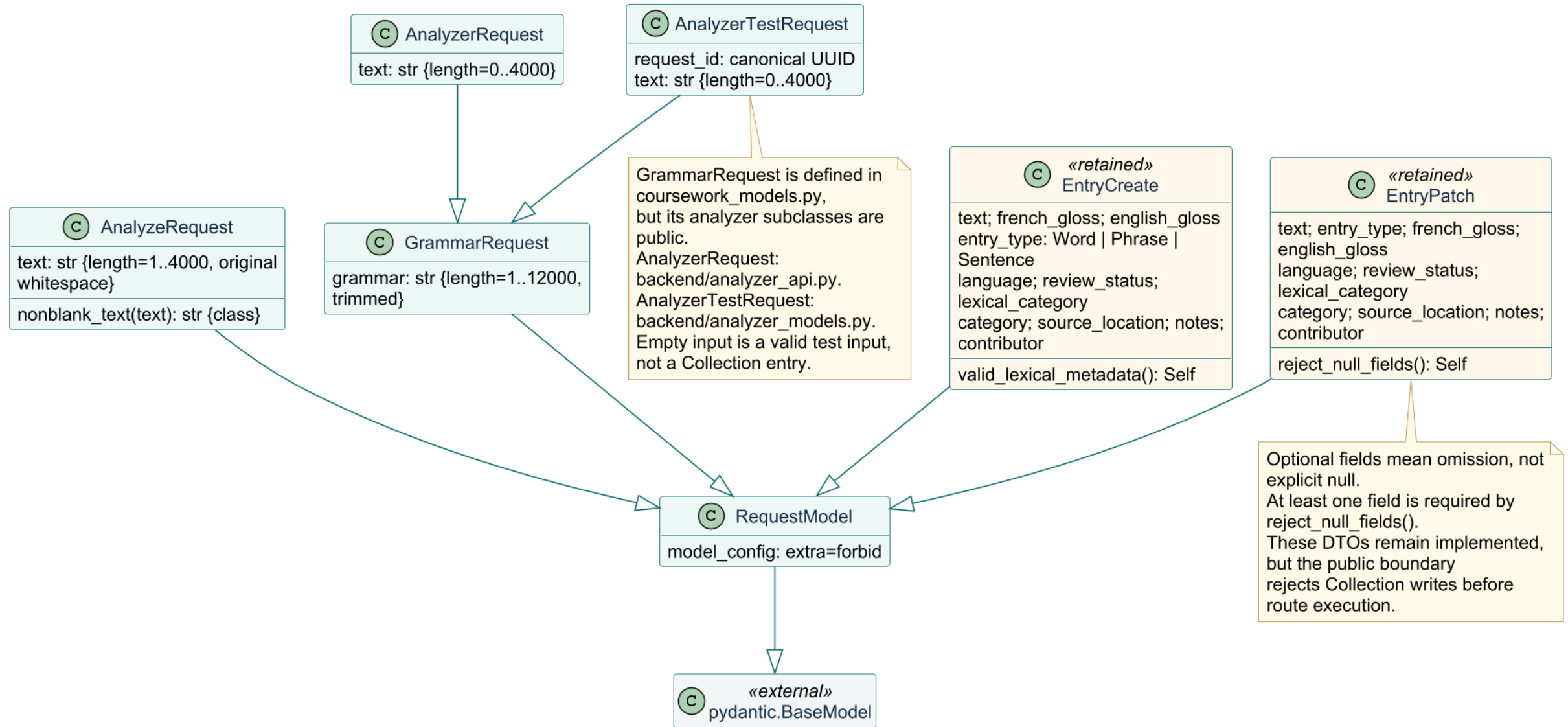
analyzer_history.test_report(offset, limit)
 «function»
 streams every saved test, pages summaries



Summaries are paged; totals/statistics use all retained tests. Repeated intentional tests remain separate observations. Collection counts are not this report's token statistics. Empty totals have `acceptance_rate=null`, not a fabricated percentage.

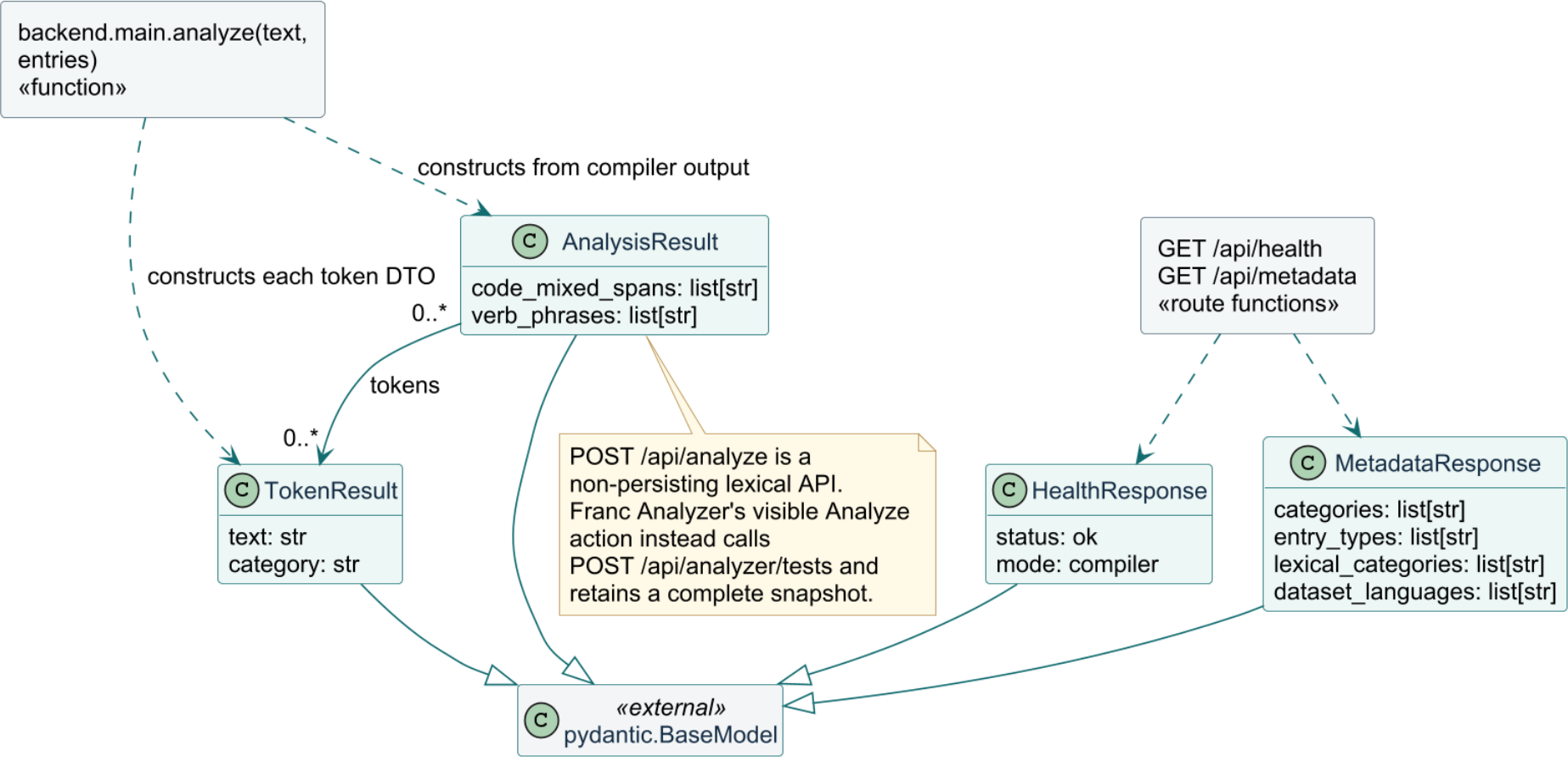
Source: backend/analyzer_models.py; aggregation: backend/analyzer_history.py.
 A report retains nested DTO references. SQL records are not model-object compositions.
 Public summaries redact ownership; vocabulary accepted/error and grammar_accepted/grammar_error stay distinct.

API inputs — current analyzer contracts and retained Collection mutations



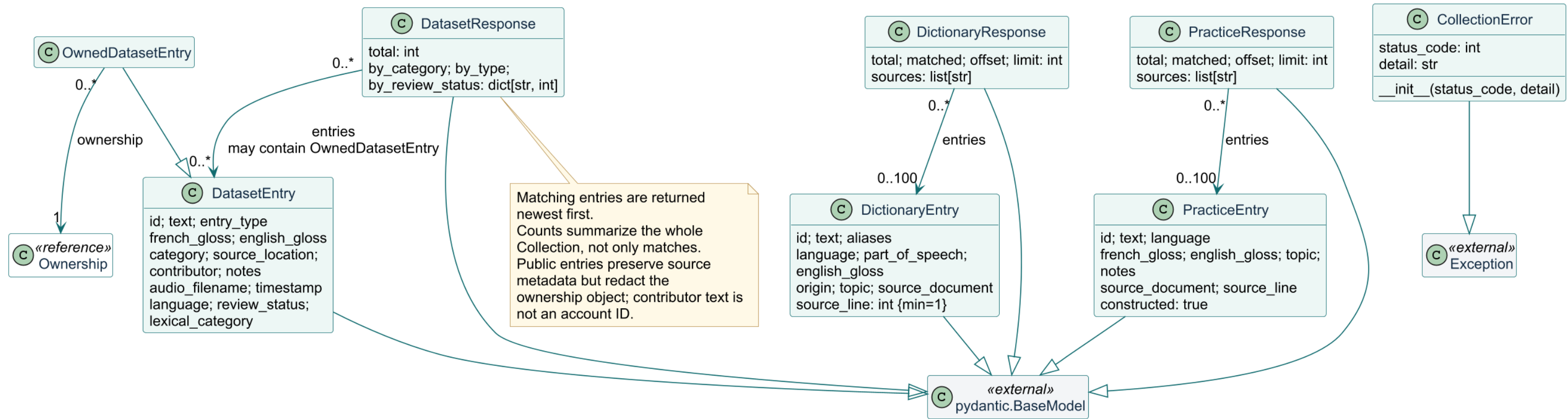
Other classes here: backend/schemas.py. Pydantic validation/serialization methods are inherited. The public analyzer uses the saved grammar; an arbitrary client grammar is not an editable public draft. Inheritance carries no multiplicities; no invented HTTP controller/request service class is introduced.

Small public API results — lexical preview, health and metadata



Source: backend/schemas.py. No selected authored methods: fields are the authored contract. BaseModel supplies serialization/validation; no invented constructors or business methods are listed. These DTOs are separate from strict TokenSnapshot and RecordedTest in analyzer_models.py.

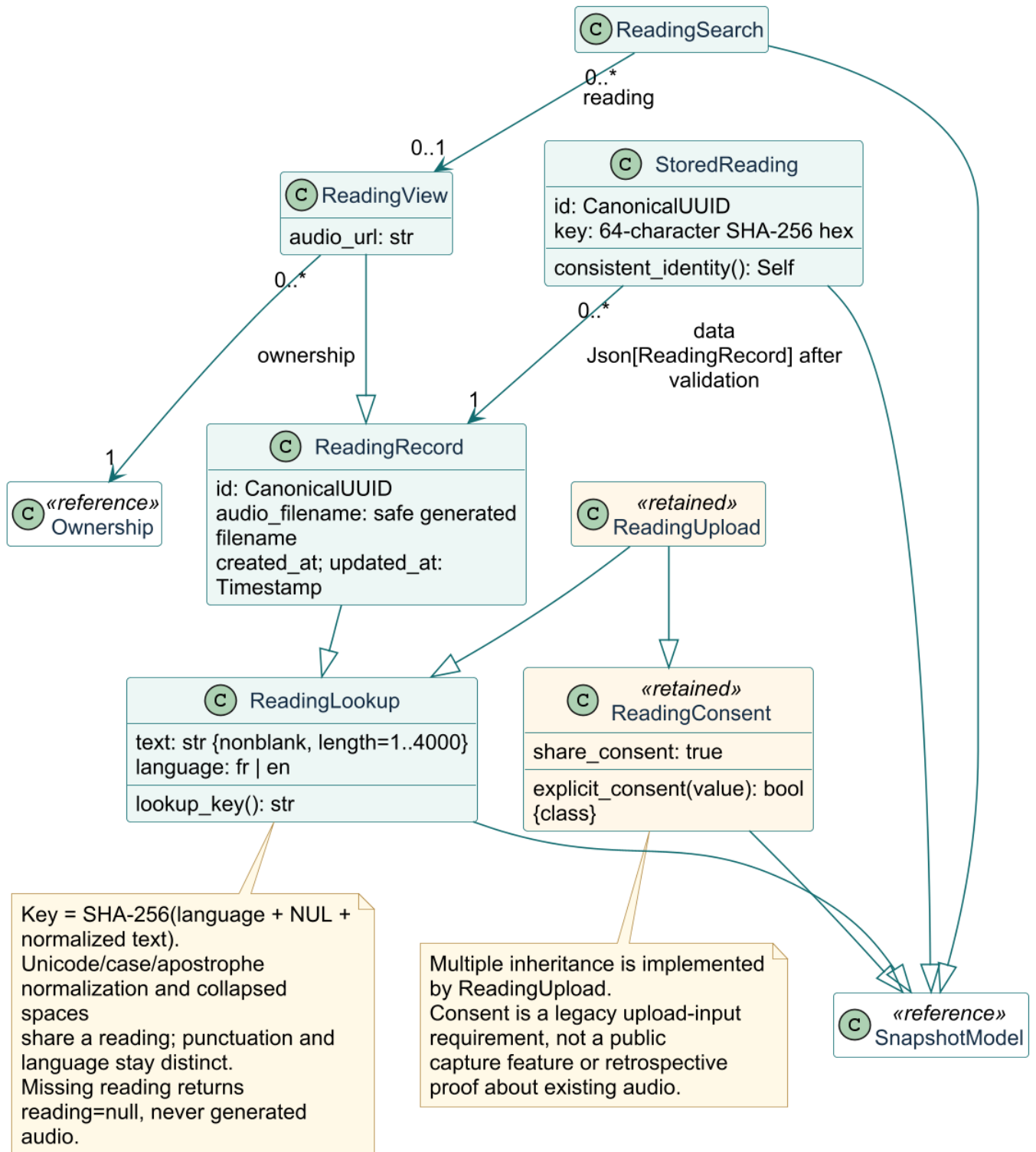
Collection, Dictionary and Synthetic examples — separate data models



Matching entries are returned newest first.
Counts summarize the whole Collection, not only matches.
Public entries preserve source metadata but redact the ownership object; contributor text is not an account ID.

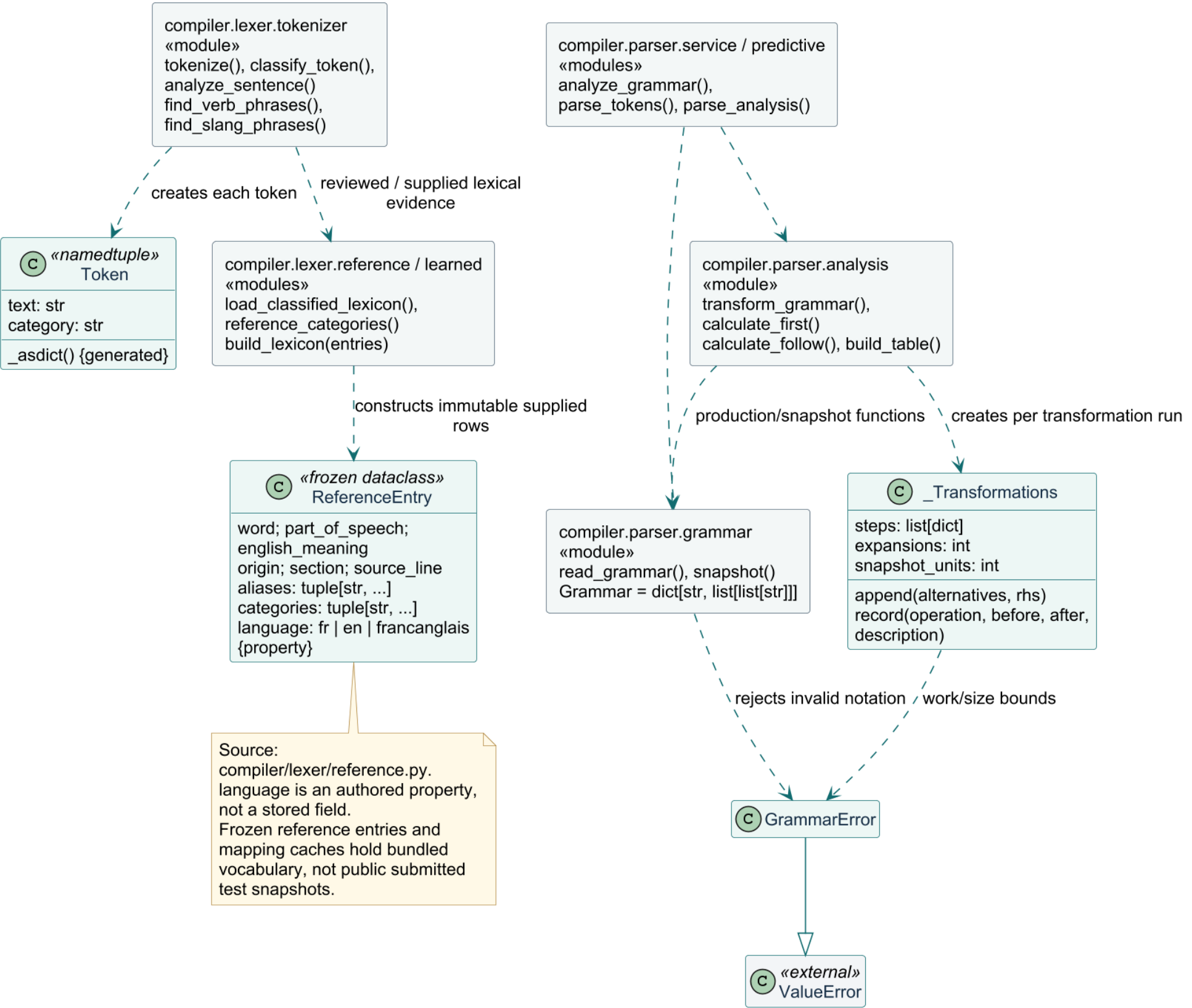
DTO source: backend/schemas.py; CollectionError: backend/collection.py.
No association links a dictionary/practice entry to a Collection entry: selecting text does not save it.
Public views are read-only. review_status records a review; it does not certify human provenance.
Dictionary/practice serialization is inherited from BaseModel, not invented business methods.

Recorded readings — public lookup/playback and retained upload contracts



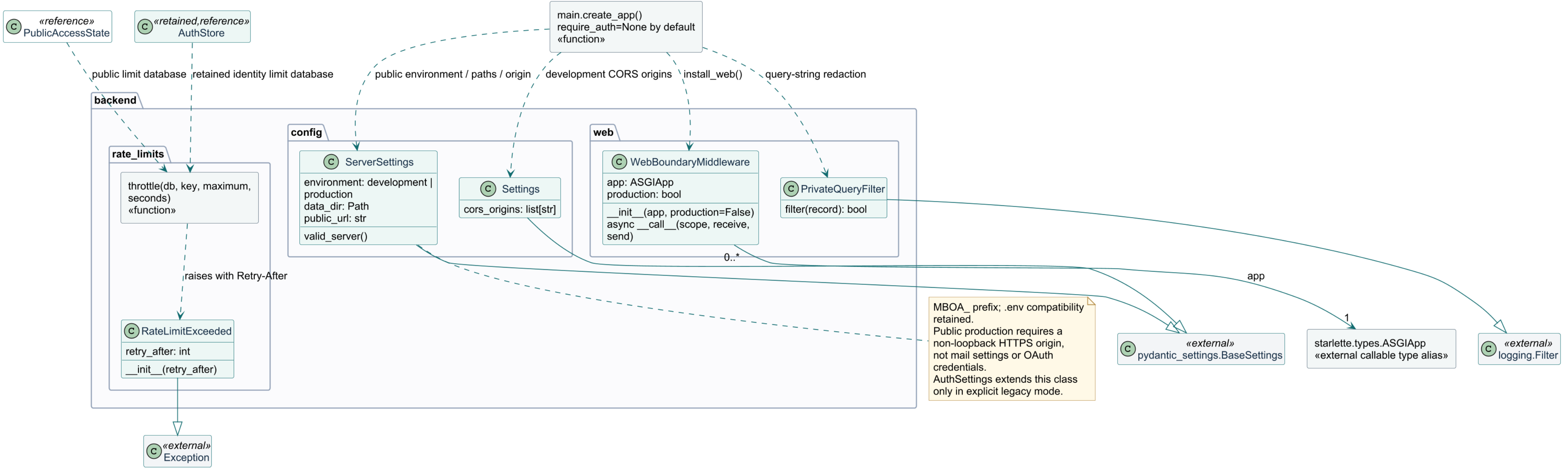
Source: backend/readings_models.py. Public routes: POST /api/readings/lookup; GET /api/readings/{id}/audio. Existing audio may be played/downloaded. Public create/replace/delete are denied. No browser TTS, microphone capture, automatic transcription or cloned voice is in this public workflow.

Compiler runtime — four authored classes, function-based algorithms



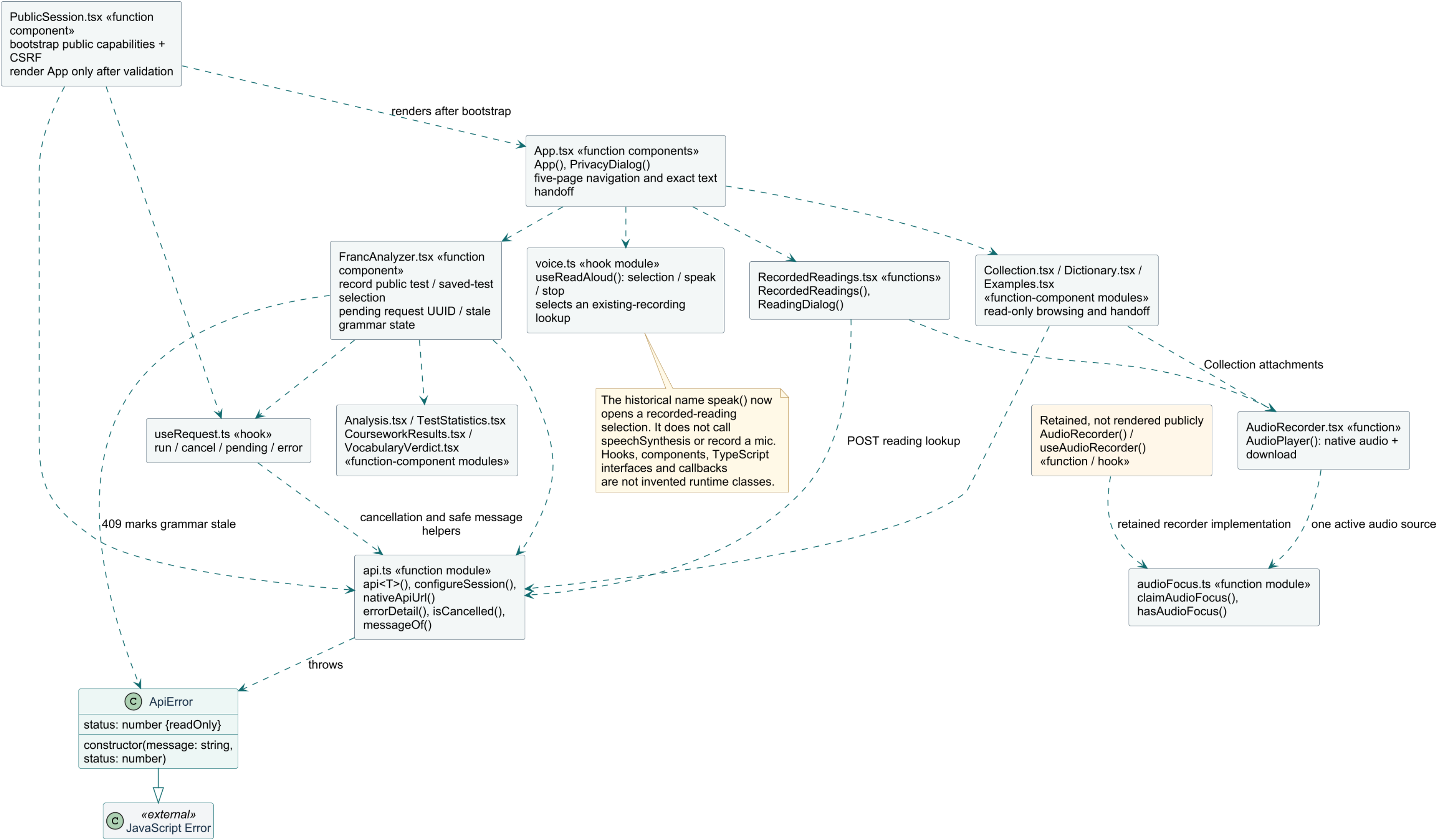
Token: compiler/lexer/tokenizer.py; GrammarError: compiler/parser/grammar.py;
_Transformations: compiler/parser/analysis.py. No LexerService, Parser or Grammar runtime class exists.
The general teaching starter has a JSON analysis cache; custom/saved corpus grammar is analyzed afresh.
Raw text stays intact; phrase annotations do not replace or collapse lexical tokens.

Runtime settings, web boundary and shared throttling



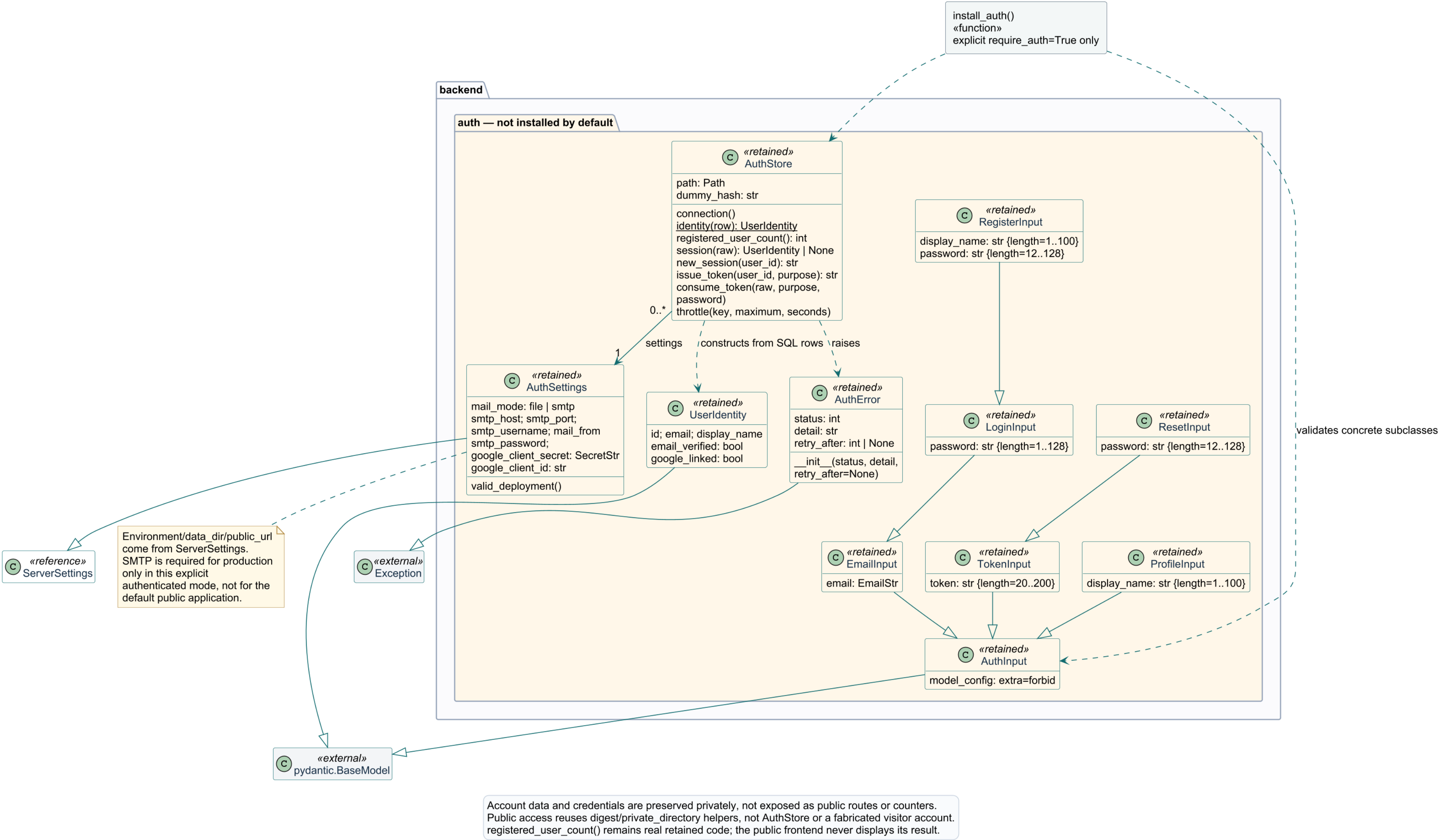
The ASGI reference is an external callable type alias, not an invented runtime class.
Settings and ServerSettings are independent BaseSettings subclasses.
Web boundary applies in all modes; public policy is in public_access.py.
A microphone Permissions-Policy allowance does not create a public recording feature.

React implementation — one authored class, functions and hooks



AuthGate, account controls, project selectors and account counter are absent from the current frontend. Only frontend.src.api.ApiError is an authored production TypeScript class. Existing AudioRecorder/useAudioRecorder source does not imply a public capture/upload capability.

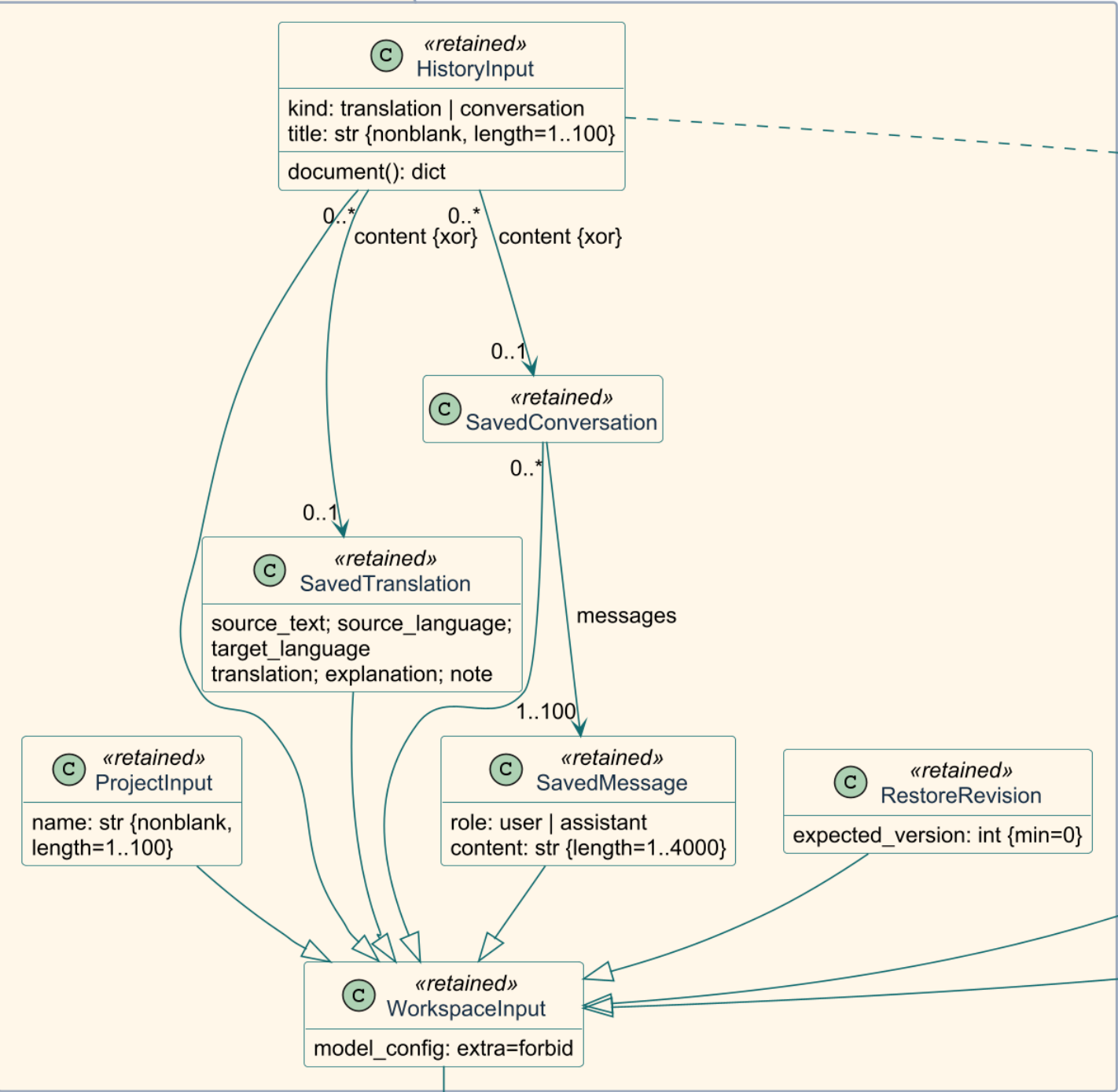
Retained accounts — explicit legacy mode only



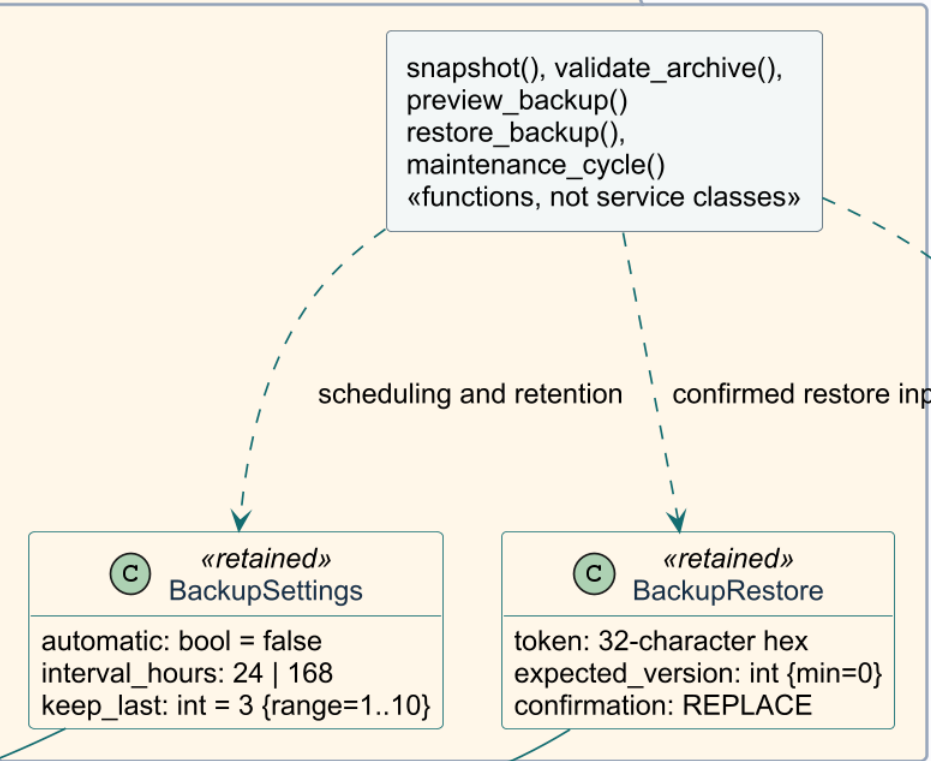
Retained workspace inputs, old saved history and operator backups

backend

workspaces — retained inputs/history



workspace_backups — operator / retained API



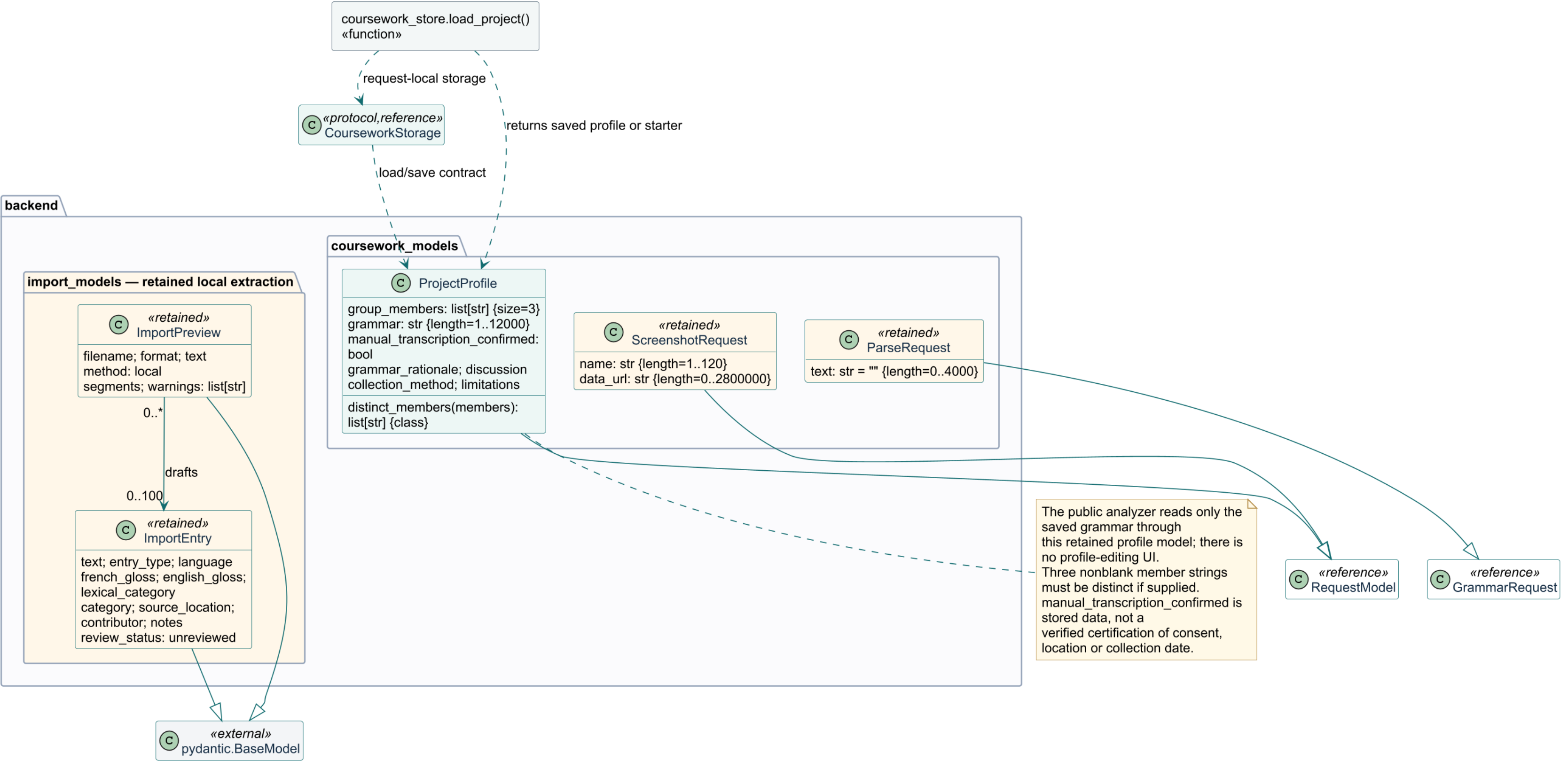
Exactly one content alternative matches kind. Historical saved translations/conversations remain data, not evidence of active translation/chat generation.

WorkspaceStore

«external»
pydantic.BaseModel

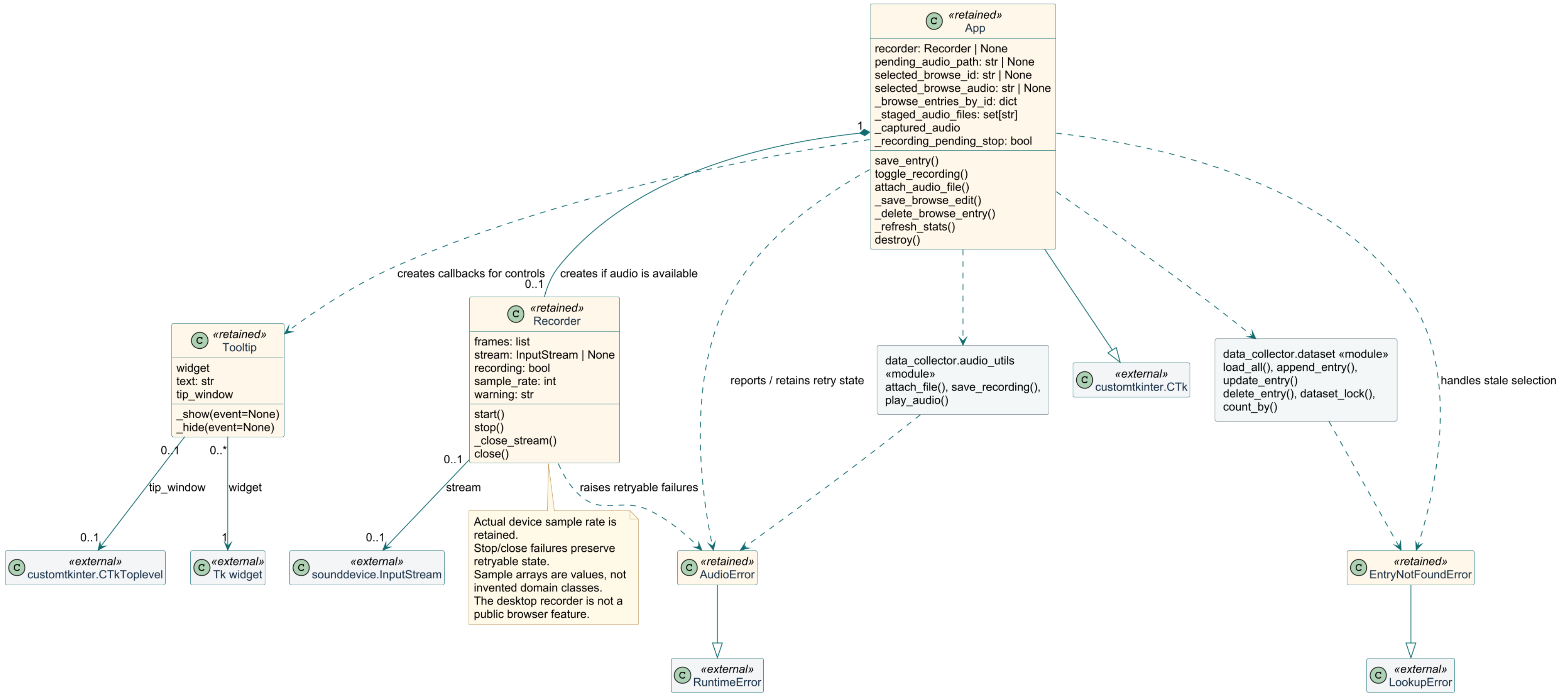
All workspace/history/import/backup HTTP endpoints are unavailable in the default public application. Automatic backup maintenance may still run privately when configured; no public recovery UI is mounted. Shared export is format 5 (format 4 compatible); private WorkspaceStore export is format 3. Older private formats 1/2 normalize during validation. ZIP manifest remains format 1.

Retained coursework/import contracts — saved grammar reuse, no public export UI



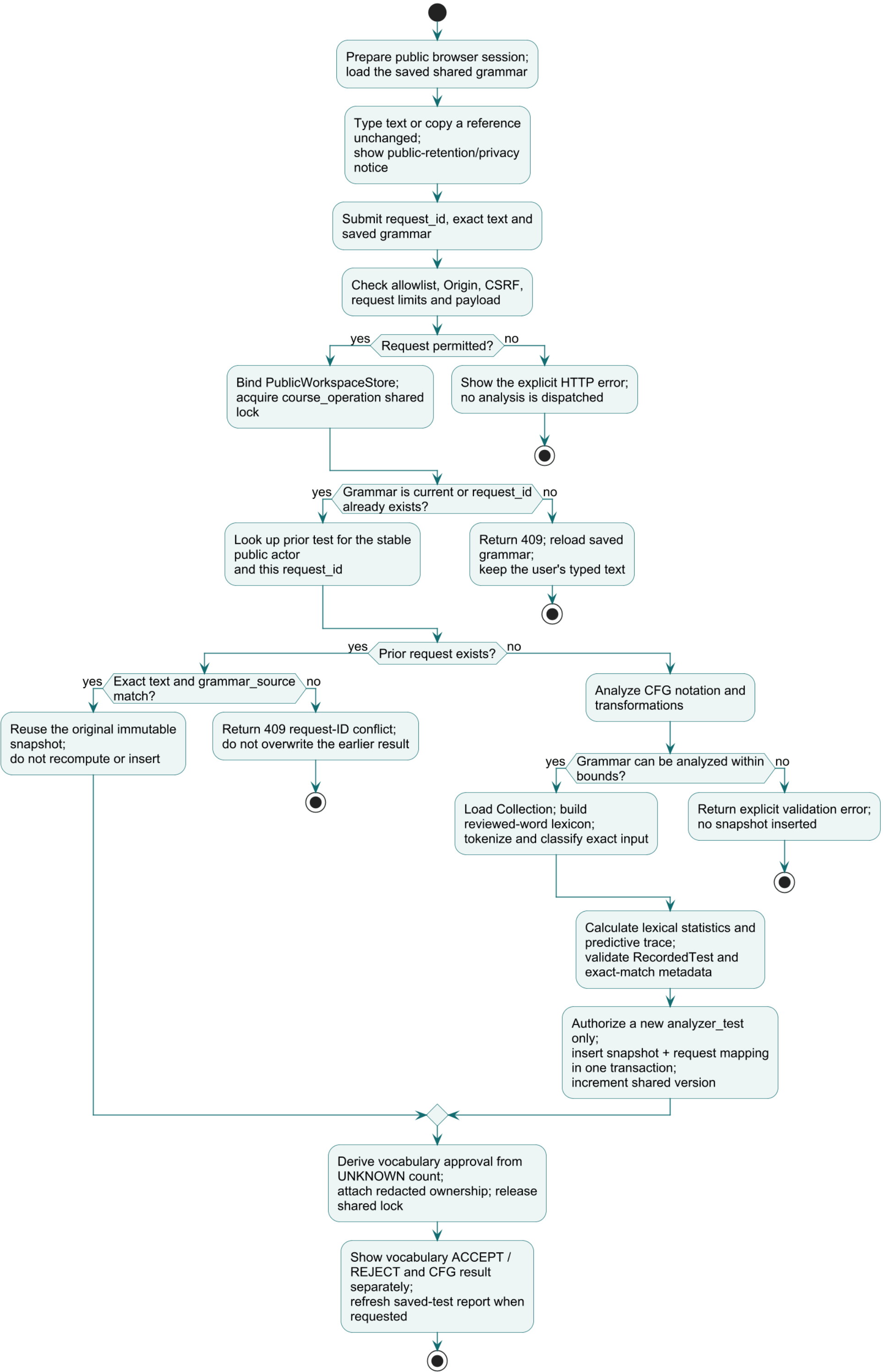
ProjectProfile still participates in active grammar loading; the other authored classes here are retained compatibility. GrammarRequest is a shared public analyzer input base, detailed in class-api-requests. Import preview, screenshots, coursework editing and generated report/presentation routes are not mounted publicly. Profile/screenshot/archive preservation does not expose legacy data through the default public application.

Retained desktop collector — actual GUI, helper and recorder classes



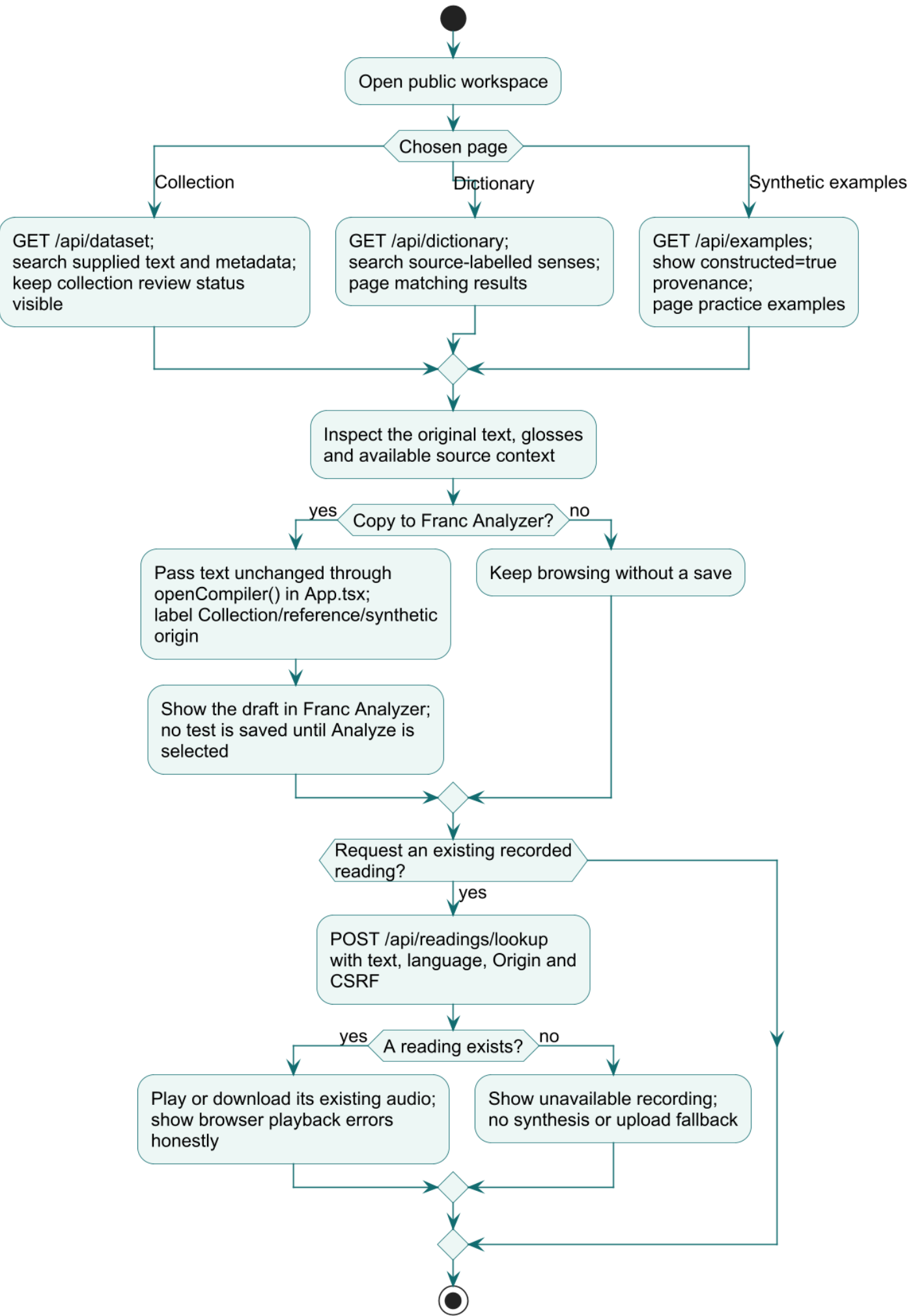
App/Tooltip: data_collector/App.py; Recorder/AudioError: audio_utils.py; EntryNotFoundError: dataset.py.
App creates the optional Recorder; bound widget callbacks retain Tooltip objects, not an App tooltip collection.
CSV and local capture workflows are retained operator/desktop compatibility, not the default web product.

Public Analyze — saved grammar, immutable tests and separate verdicts



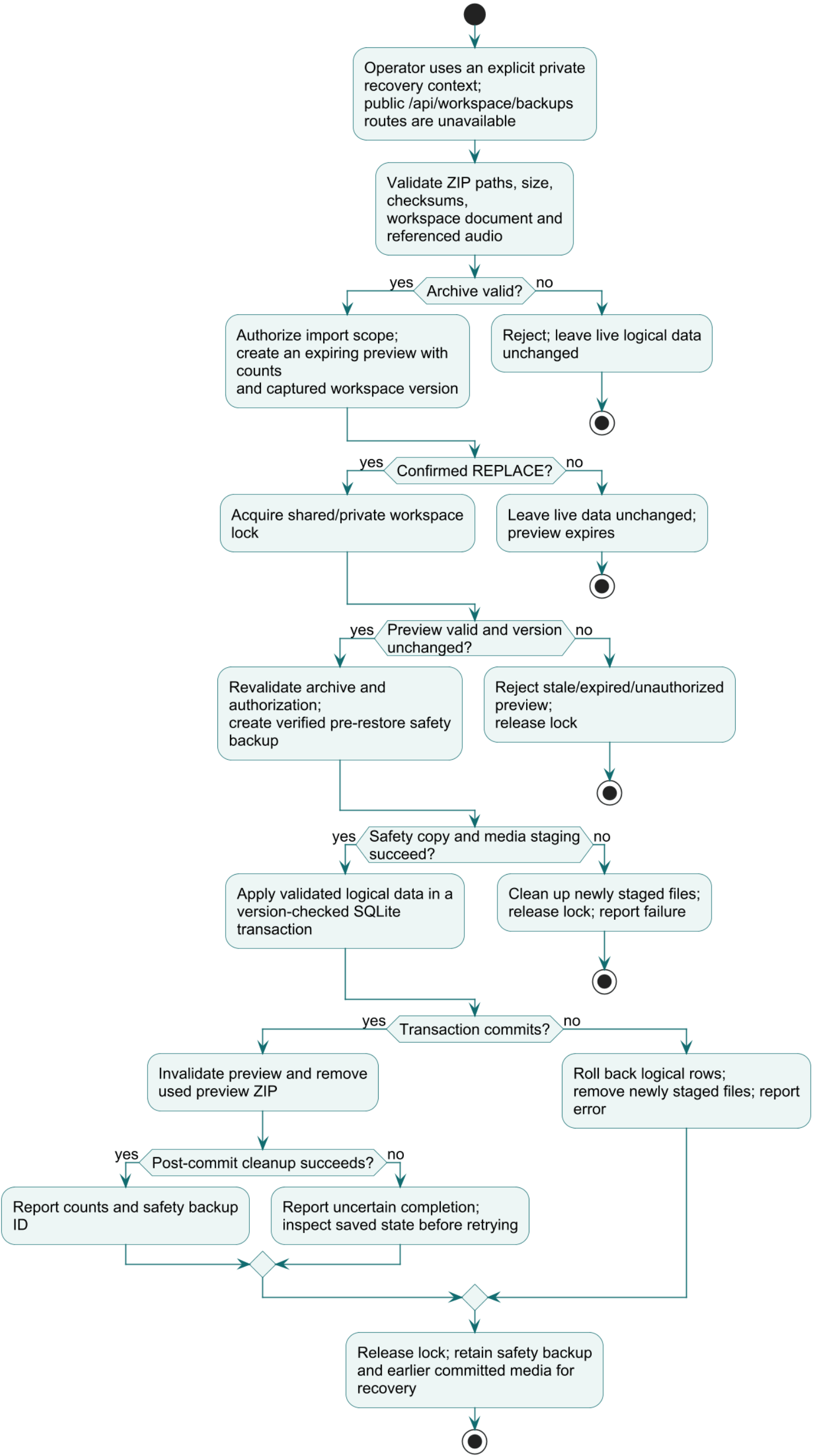
Analyze in the UI records a public test; Collection is never mutated by this workflow.
A missing CFG table cell, non-LL(1) grammar or parser bound can yield a retained rejected test.
Stopping client-side waiting does not cancel or roll back a server commit; inspect saved tests before retry.
An existing identical retry may return its old snapshot even after offline grammar curation.

Read-only reference use — browse, inspect and hand off without mutation



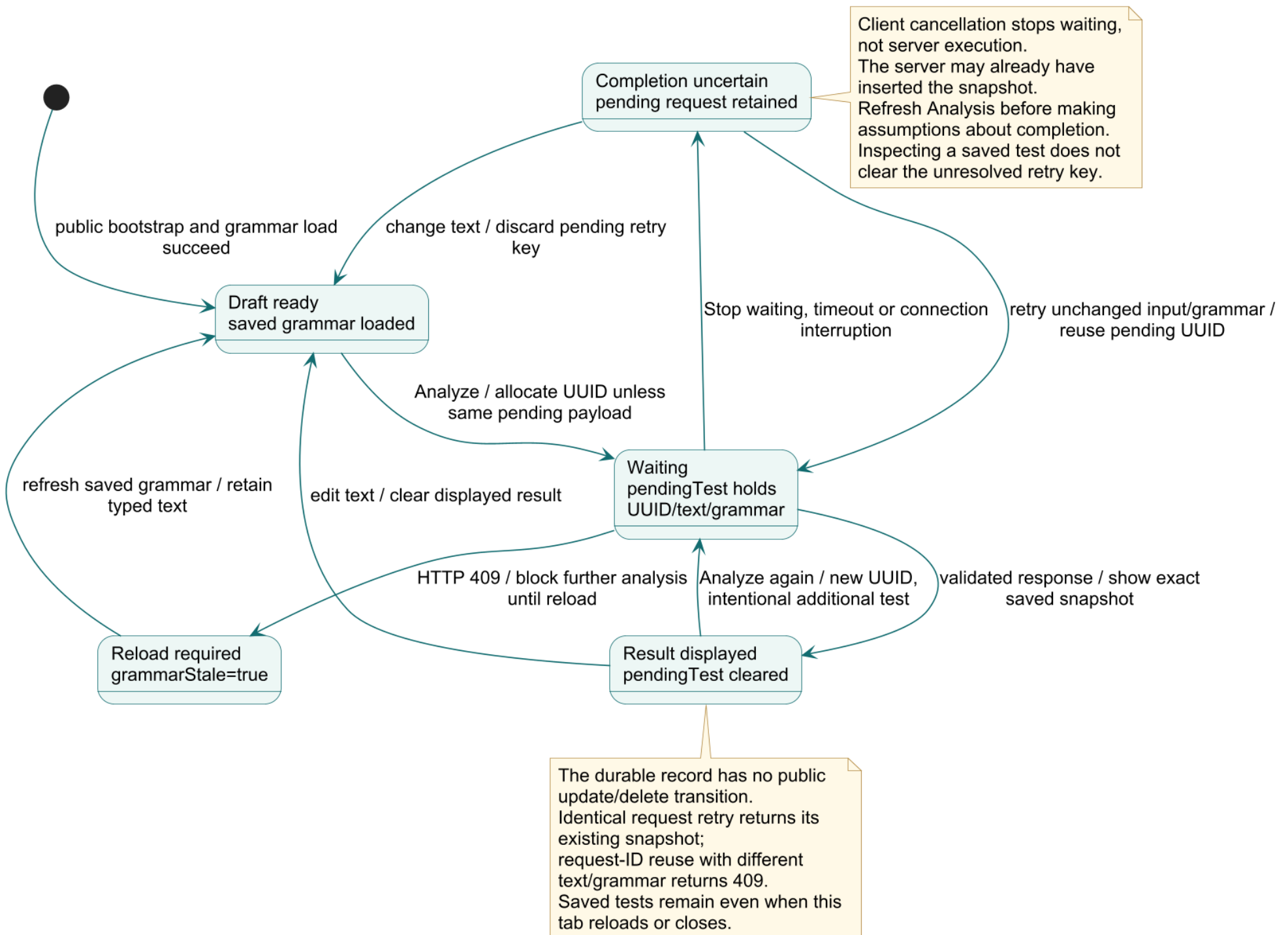
The sources are separate datasets, not subclasses of one collected-statement entity.
Human collection place/date/consent are unconfirmed; a label or review flag is not fieldwork certification.
No Collection, grammar or audio editing occurs; browsing and handoff never add an entry or a test.

Retained restore procedure — private operator/legacy workflow only



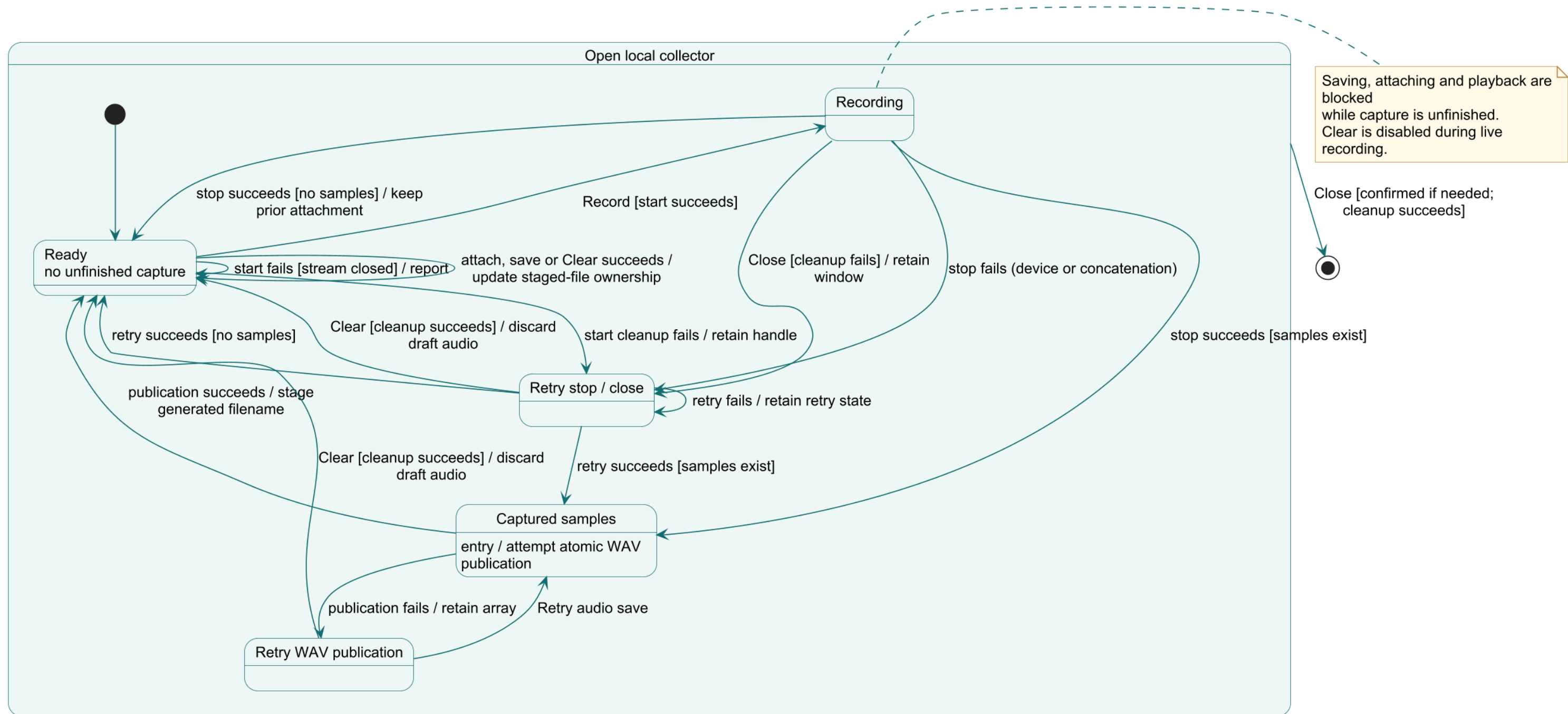
Shared format 5 includes ownership, request mappings, archived profiles and readings; format 4 remains compatible. Private formats 1/2 normalize to format 3; outer ZIP manifest is format 1. Non-system legacy restore cannot change another creator's records or original archived profiles. PublicWorkspaceStore.authorize_import() always rejects; no browser restore feature is implied. Filesystem staging and SQLite are coordinated operations, not one cross-file ACID transaction.

Public test submission — browser state versus durable result



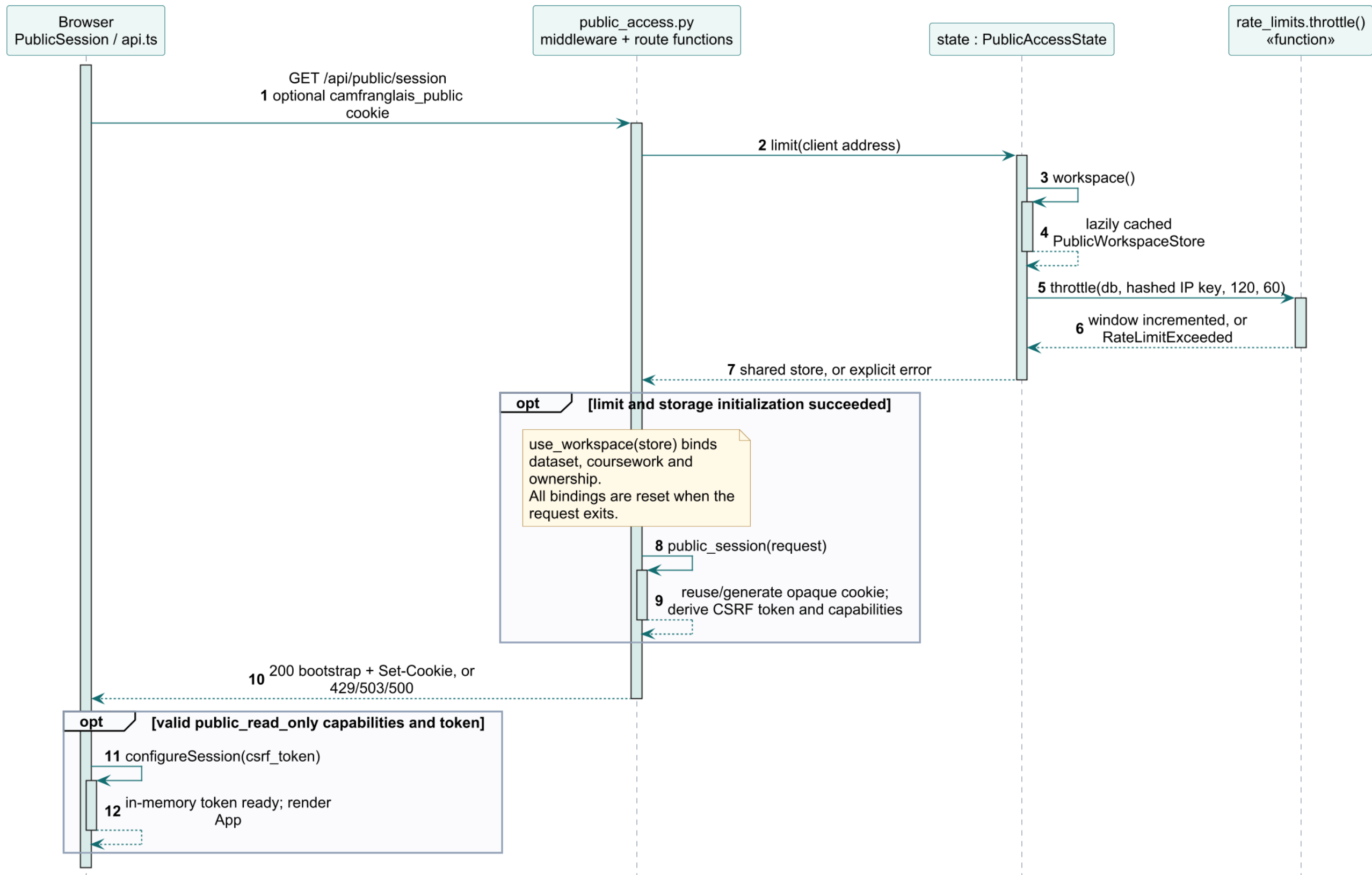
Derived logical browser states, not an invented enum or application class.
Source: frontend/src/FrancAnalyzer.tsx, api.ts, useRequest.ts; persistence: analyzer_history.py/public_access.py.
Selecting an old snapshot does not rerun it against today's Collection or grammar.

Retained desktop only — recorder readiness and retry states



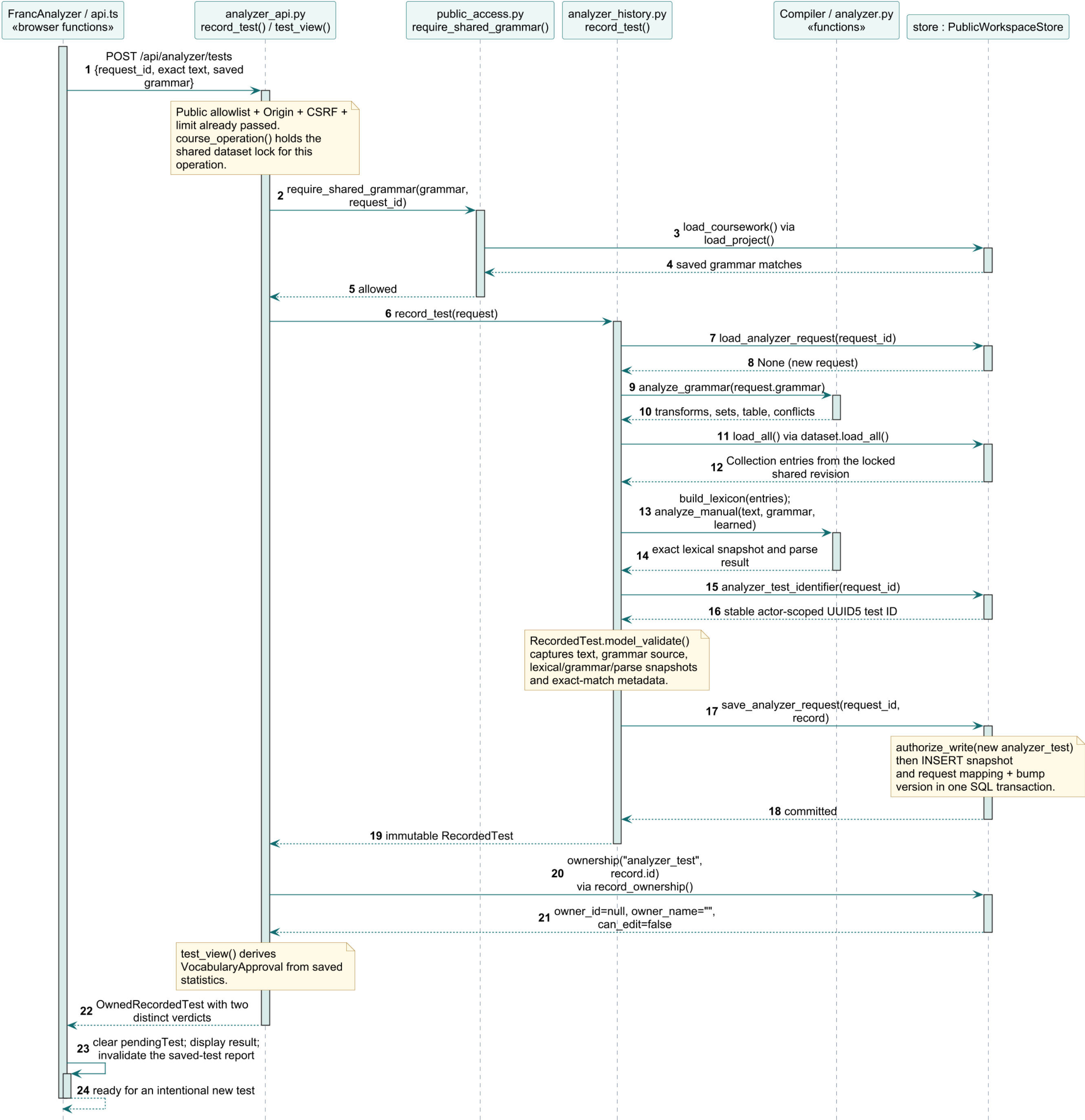
Source: data_collector/App.py and audio_utils.py. This is not a public browser microphone feature. Logical states derive from recording, stream, pending-stop and captured-array fields, not invented classes/enums. Ready may include a staged filename. Cleanup failures preserve the retry state rather than pretending to close. Draft-owned files may be discarded; committed recordings are not owned by the draft.

Public bootstrap — CSRF preparation without an account



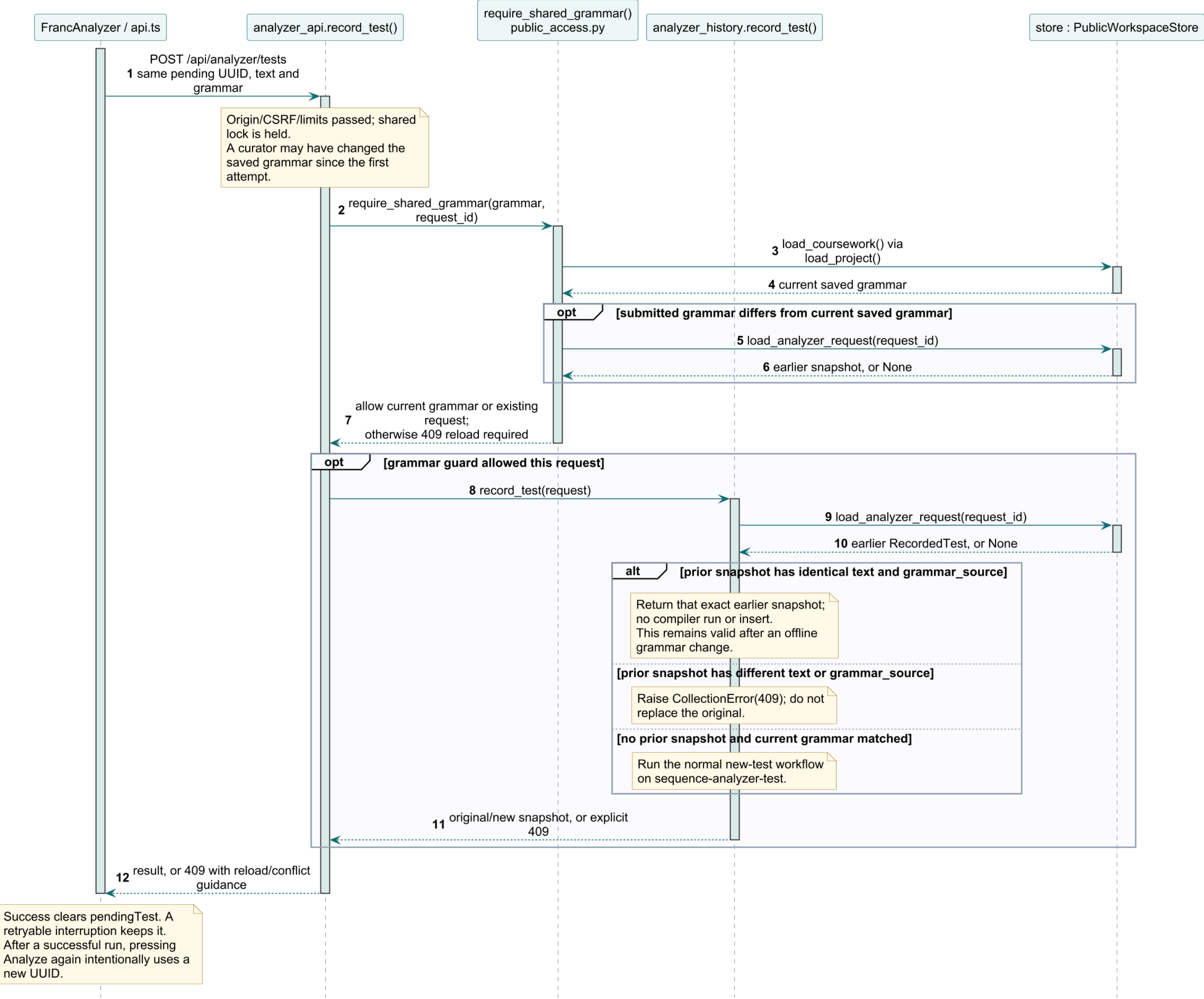
Successful capabilities: analyze/save_tests=true; edit_collection/edit_grammar/edit_recordings=false.
 Cookie: HttpOnly, SameSite=Lax, Secure in production. It is CSRF state, not login or user identity.
 Rate-limit rows live in public-access.sqlite3; no auth.sqlite3 initialization or fake account is required.
 Health bypasses the shared limit; this sequence shows the default include_academic=true bootstrap.

Analyze and retain one new public test — successful path



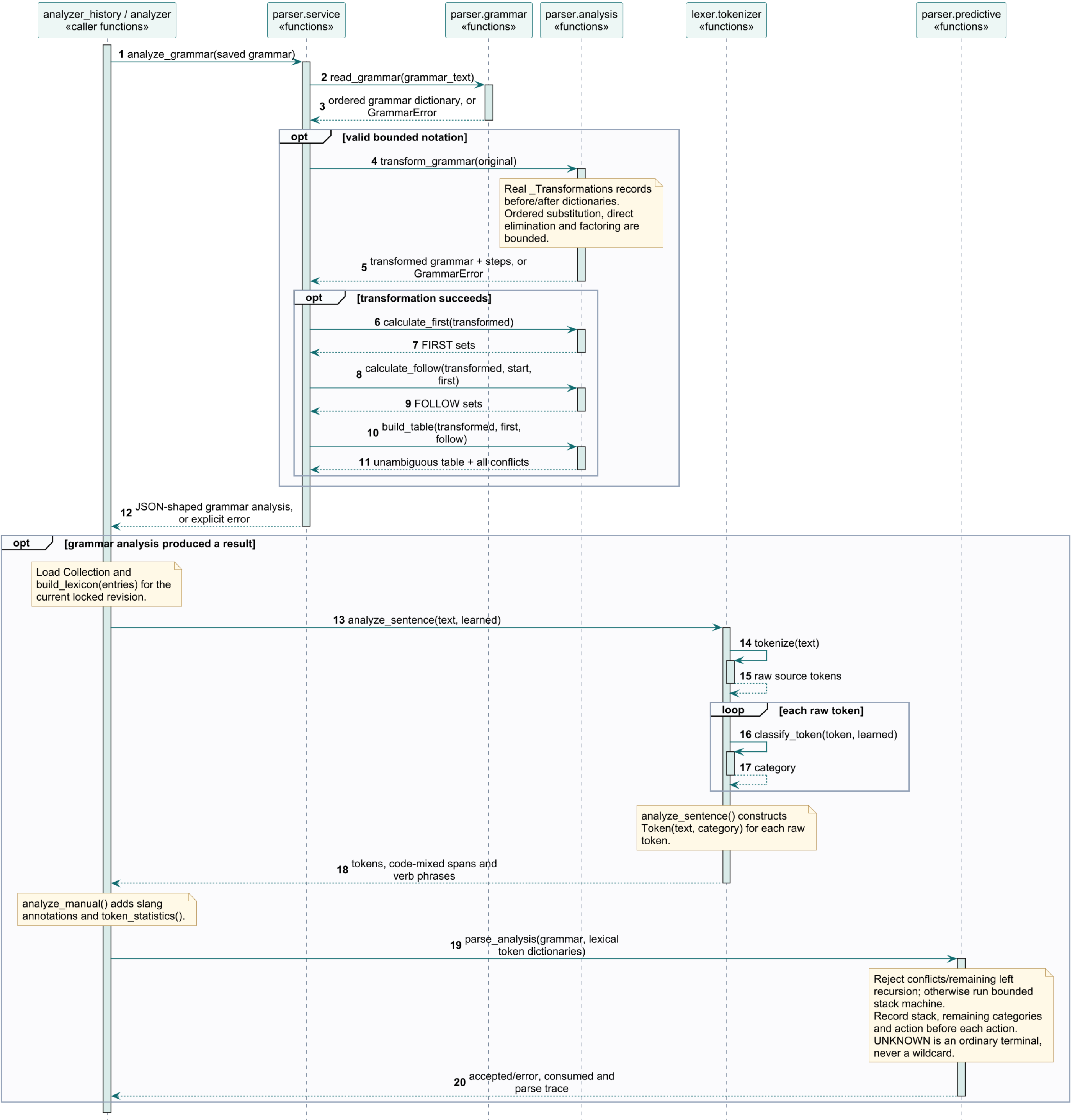
Storage calls abbreviate the real dataset/coursework_store/ownership ContextVar adapters. A retained grammar rejection is still a completed test; malformed/oversized grammar raises before insertion. No Collection entry, grammar, recording or private history is modified by this operation. Cancellation/timeout cannot promise rollback; identical retries are detailed separately.

Test retry — preserved snapshot versus stale/new request



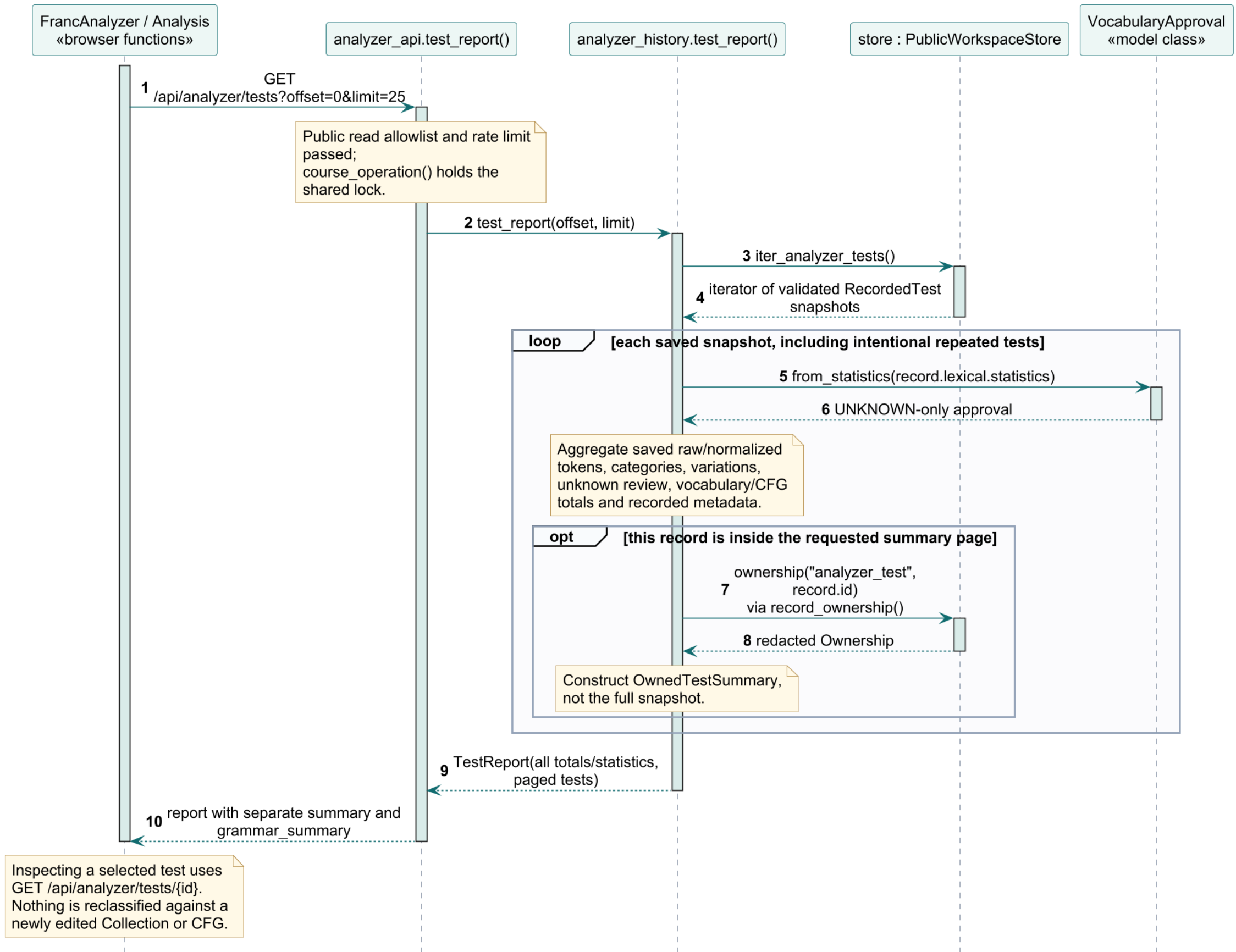
Stable PUBLIC_OWNER_ID makes retry lookup survive cookie changes; it is not a fabricated account. Public request IDs are idempotency keys, not credentials. Exact payload comparison prevents key reuse from overwriting data. A new stale request fails before analysis; existence alone does not bypass the later exact-payload check.

Deterministic compiler — actual function calls and trace construction



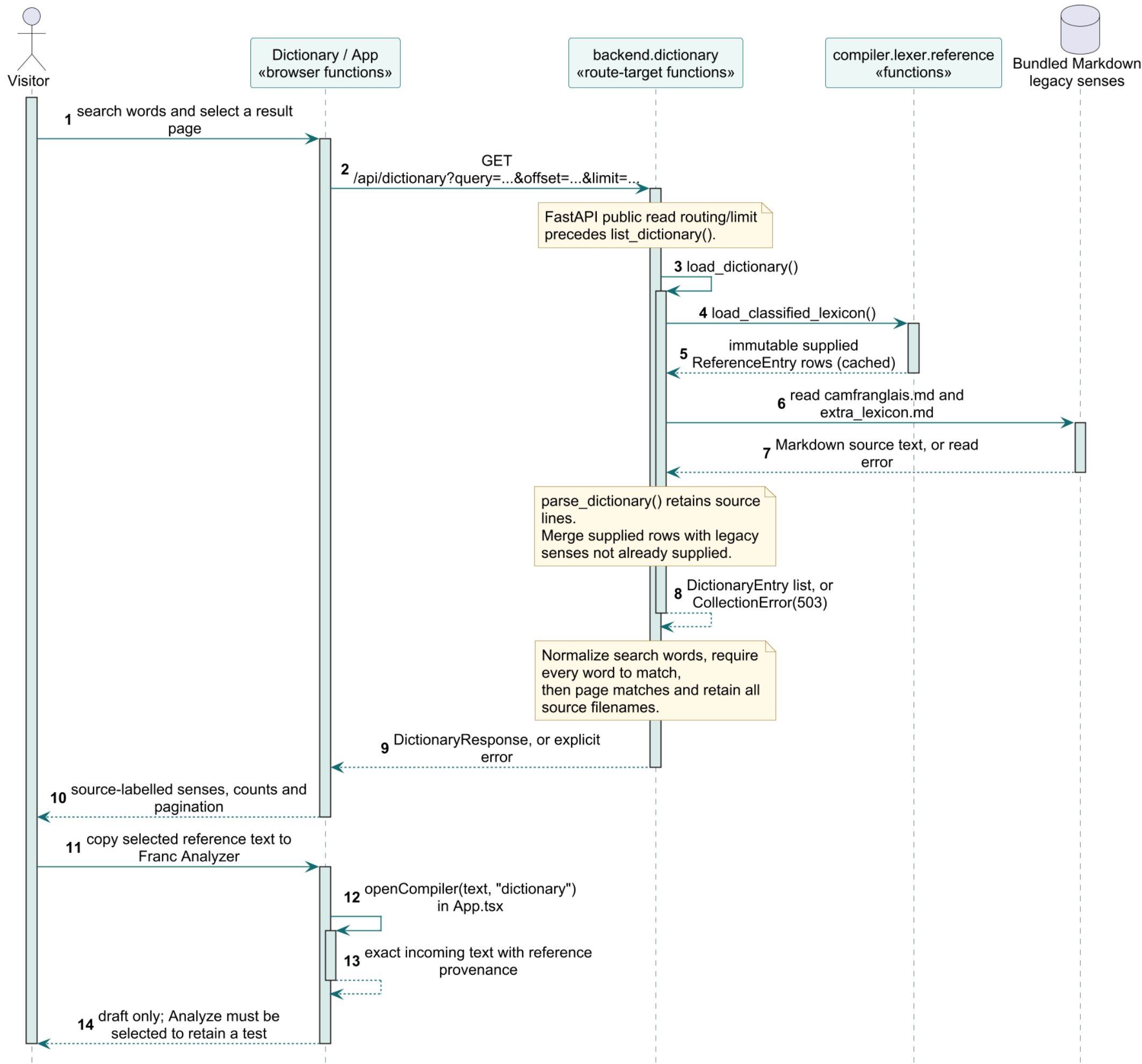
This shows the saved/custom grammar path; the generic teaching starter alone has a decoded JSON cache. Parser bounds: 256 input tokens, 1000 characters/token, 2000 trace steps, 1024 stack symbols. Vocabulary approval is a separate later calculation: only UNKNOWN causes that verdict to reject.

Analysis statistics — all saved snapshots, one page of summaries



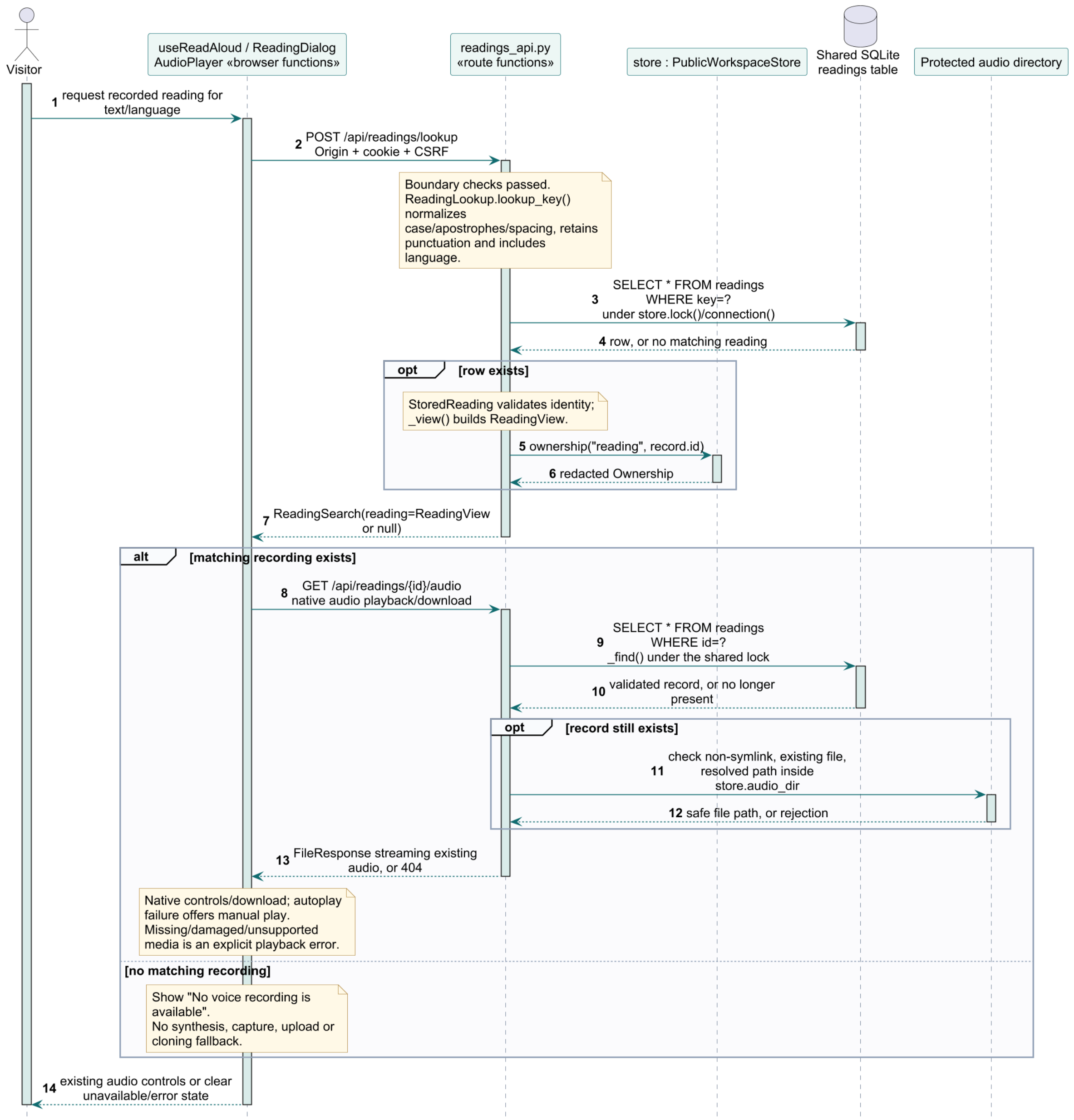
Iteration is lazy: the store validates SQL rows as consumed, retaining one complete snapshot at a time. offset/limit page only summaries, not the population underlying the aggregates. Empty acceptance rates are null; historical test counts are dated observations, not permanent constants.

Dictionary search and exact text handoff — no Collection write



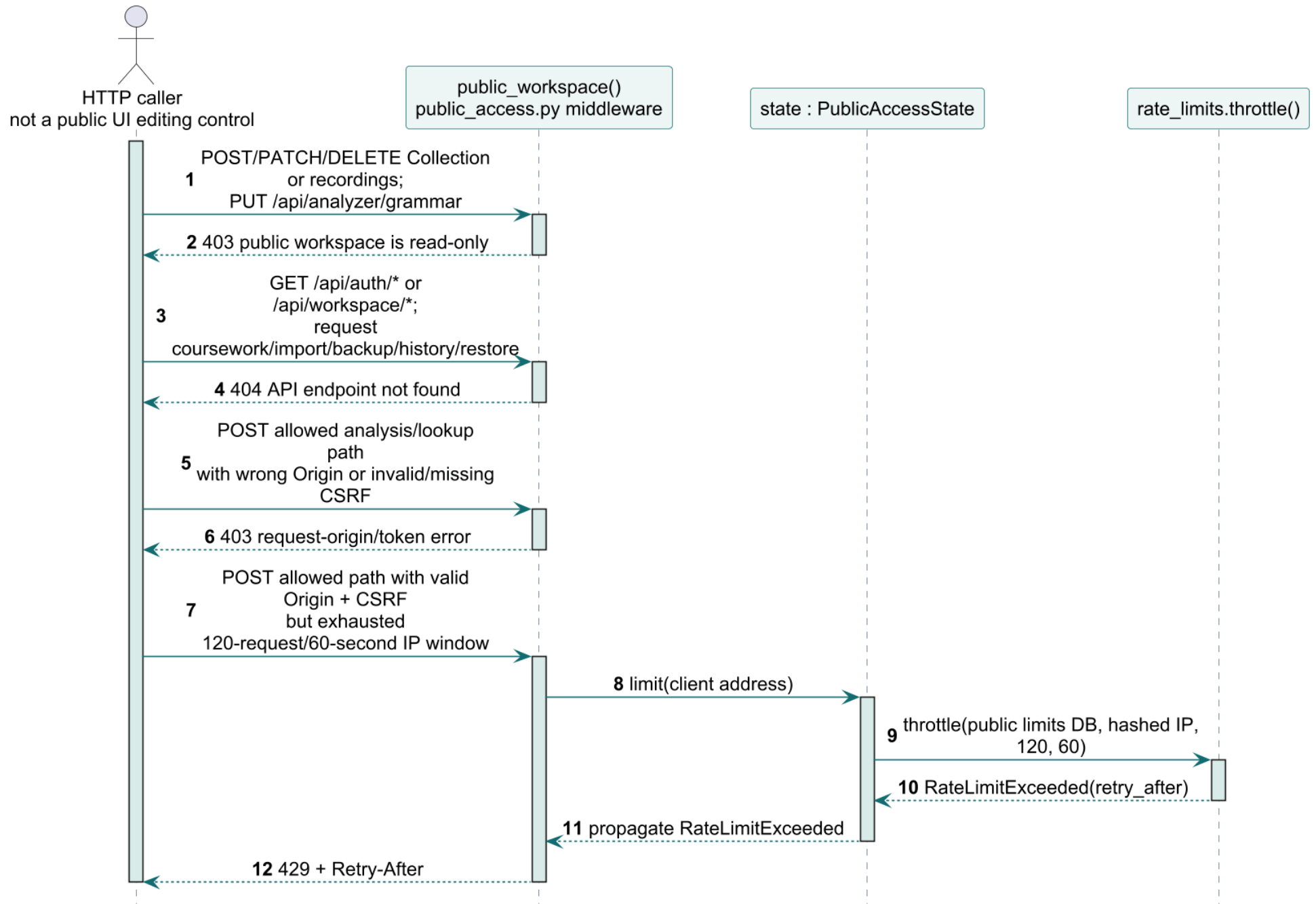
Dictionary reference data is not Collection and is not evidence of human collection.
The UI handoff sends no POST and never inserts a Collection entry.
The 938-entry merged dictionary is a dated source snapshot, not a test count or fieldwork result.

Existing recorded reading — lookup, guarded audio and honest absence



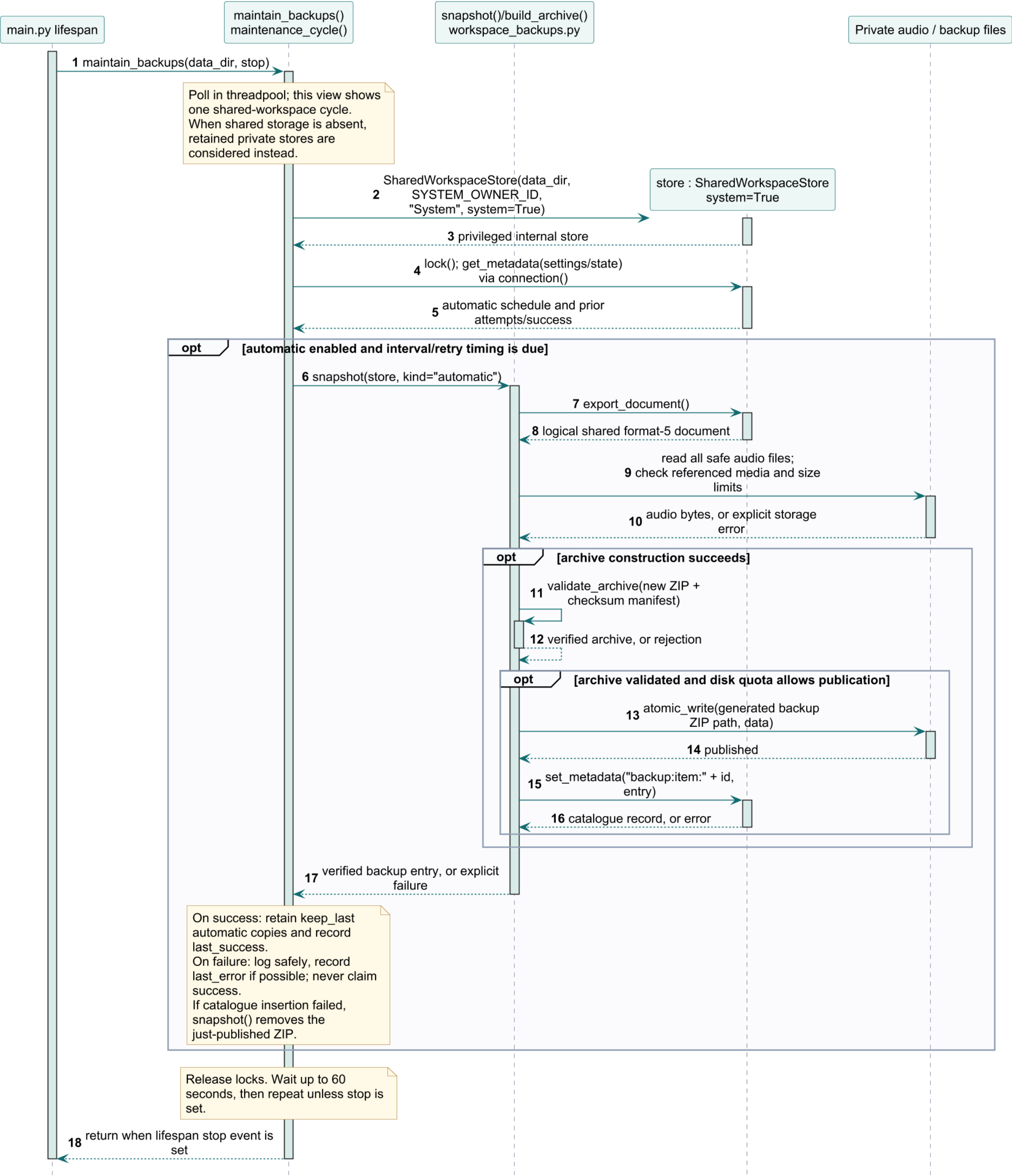
The browser hook name speak() selects a saved-reading lookup; it does not synthesize speech. Public ownership redaction never rewrites historical creator records. Collection attachment playback uses the separate read-only GET /api/dataset/{id}/audio route.

Public policy enforcement — rejected writes and hidden legacy routes



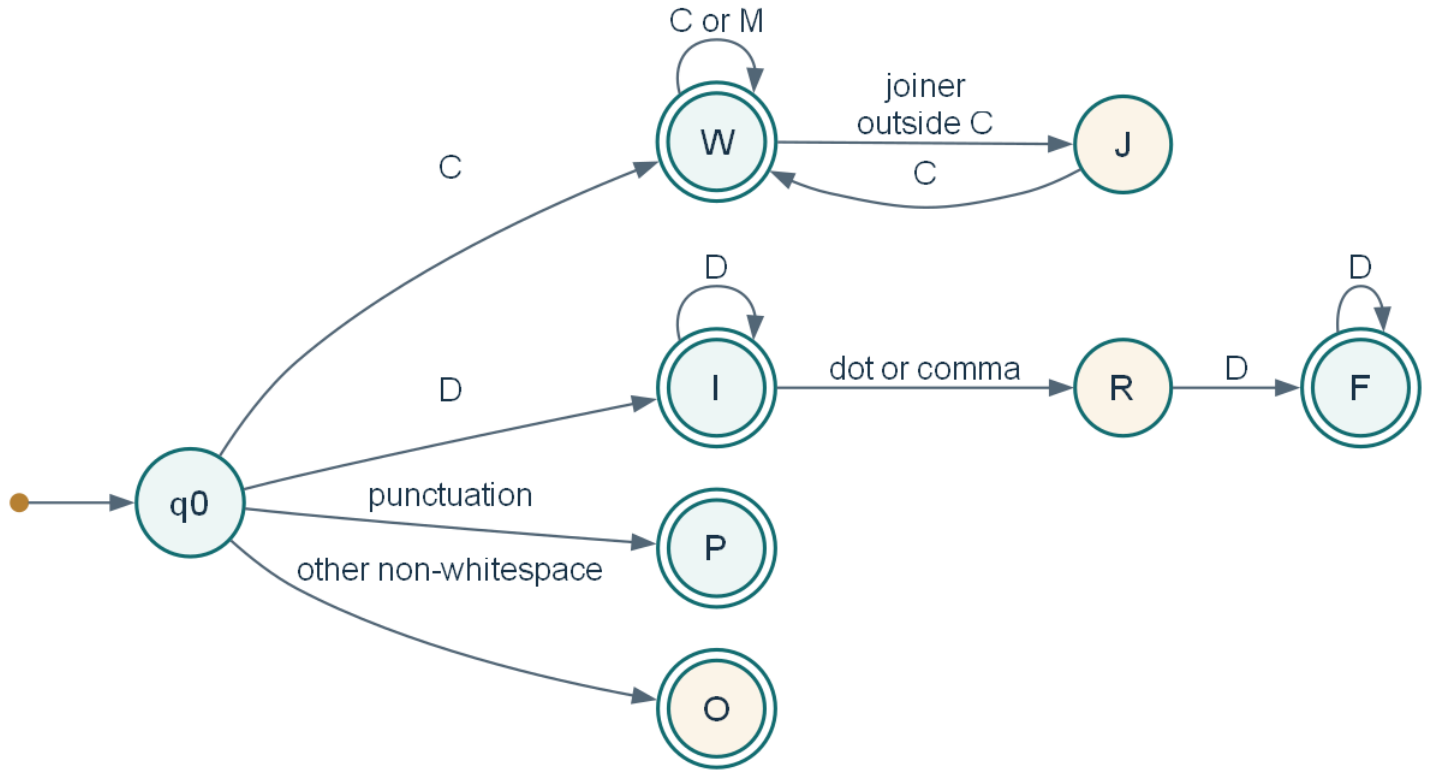
Allowed POST paths: `/api/analyze`, `/api/analyzer/analyze`, `/api/analyzer/tests`, `/api/readings/lookup`.
 Read-only-resource path checks distinguish forbidden writes (403) from absent legacy endpoints (404).
 These failures dispatch no domain mutation. PublicWorkspaceStore also rejects forbidden writes/imports.
 This sequence depicts separate requests; every call/reply and activation is independently balanced.

Operator background backup — one due shared-workspace cycle



No public backup API or recovery screen is mounted; maintenance can run privately in public mode. Default automatic=false; configured intervals are 24/168 hours and keep_last is 1..10. This backup preserves workspace/media, not auth.sqlite3 or /etc/camfranglais.env. Archive validation and atomic file publication precede a successful catalogue entry.

LEXER / FINITE-STATE TOKEN RECOGNITION



PARSER / DETERMINISTIC PUSHDOWN CONTROL

